A RAG-Powered Chatbot for CyberArk API Documentation

DISSERTATION

Submitted in partial fulfilment of the requirements of the
Degree: MTech in Artificial Intelligence and Machine Learning

By
Saya Venkata Thrivedi
2023AA05203

Under the supervision of
Sasikiran Varikunta
(Staff Software Engineer, CyberArk)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
Pilani (Rajasthan) INDIA
(August 2025)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

This is to certify that the Dissertation entitled "***A RAG-Powered Chatbot for CyberArk API Documentation***" and submitted by **Mr. Saya, Venkata Thrivedi**, **ID No. 2023AA05203** in partial fulfilment of the requirements of **AIMLCZG628T** Dissertation, embodies the work done by him under my supervision.



Place: Hyderabad

Date: 06-08-2025

Signature of the supervisor

Sasikiran Varikunta

Staff Software Engineer

Abstract:

This project presents the design and implementation of a Retrieval-Augmented Generation (RAG) system specifically developed to enhance developer interaction with CyberArk API documentation. The system addresses the challenge of efficiently accessing and understanding complex enterprise security API specifications through an intelligent conversational interface that provides context-aware, accurate responses to technical queries.

The methodology encompasses a multi-stage retrieval pipeline that begins with systematic document preprocessing and chunking of API documentation into semantically meaningful segments. The approach implements sophisticated query expansion techniques, generating multiple subqueries from user inputs to ensure comprehensive information coverage. The system employs hybrid retrieval strategies, combining BM25 keyword matching with semantic similarity search using FAISS vector stores and HuggingFace embeddings for optimal document selection.

A key innovation lies in the implementation of recursive retrieval-generation with self-correction mechanisms. The system generates initial responses, identifies knowledge gaps through automated analysis, and performs additional targeted retrieval to enhance answer completeness. Neighbourhood expansion techniques retrieve adjacent document chunks to maintain contextual coherence, while BERT-based reranking prioritizes semantically relevant content based on query similarity scores.

The architecture integrates comprehensive guardrails for security and compliance, ensuring appropriate content filtering while maintaining system reliability. Phoenix tracing provides detailed observability throughout the retrieval and generation pipeline, enabling performance monitoring and system optimization. The implementation includes advanced prompt engineering with role-specific system instructions tailored for technical documentation assistance.

A functional prototype demonstrates the complete pipeline through a Gradio-based web interface, supporting realistic technical queries about endpoint specifications, authentication procedures, and parameter configurations. The system successfully handles complex multi-part queries while maintaining response accuracy and technical precision through grounded generation techniques that rely exclusively on retrieved documentation context.

Table of contents:

Abstract 3
Table of contents 4
List of Abbreviations 7
List of Tables 8
List of Figures 9
Methodology 10
Chapter 1. System Architecture 10
1.1 Data Collection 10
1.2 Data Processing 11
1.3 Data Enrichment 11
1.4 Embedding Generation 11
1.5 Retrieval and Generation 12
1.6 Design Considerations and Modularity 12
Chapter 2. Data Collection and Processing 13
2.1 Data Collection 13
2.2 Data Processing and Format Conversion 13
2.3 Specification Parsing and Chunking 13
2.4 RAG-Optimized Formatting 14
2.5 Data Enrichment 14
2.5.1 Motivation for Enrichment 14
2.5.2 Automated Enrichment Pipeline 15
2.6 Structured Markdown Generation 15
2.7 Advantages of the Enrichment Approach 16
Chapter 3. Embedding Comparison and Creation 17
3.1 Motivation and Goals 17
3.2 Experimental Setup 17
3.2.1 Evaluation Process 17
3.3 Results and Interpretation 18
3.3.1 Embedding Time Comparison 18
3.3.2 Query Time Analysis 19
3.3.3 Retrieval Quality (NDCG) 19
3.3.4 Embedding Dimension Considerations 20
3.3.5 API Keyword Coverage 20
3.3.6 Composite Score and Model Recommendation 21
3.4 Saving and Deployment 21
Chapter 4. Comparative Analysis and Selection of Retrieval Strategies 22
4.1. Introduction 22
4.2 Baseline Infrastructure and Configuration 22
4.2.1 Embedding Model and Vector Store Architecture 22
4.2.2 System Prompt Architecture and Configuration 23
4.3 Retrieval Method 1: Ensemble Retrieval Search 23
4.3.1 Conceptual Framework and Implementation 23
4.3.2 Technical Implementation Details 23
4.3.3 Advantages and Performance Characteristics 24
4.3.4 Limitations and Considerations 24

4.4 Retrieval Method 2: Neighbourhood Expansion with Context Aggregation..... 25
4.4.1 Theoretical Foundation and Motivation..... 25
4.4.2 Implementation Architecture and Process Flow..... 25
4.4.3 Context Window Management and Optimization..... 26
4.4.4 Advantages and Performance Benefits..... 26
4.4.5 Limitations and Technical Challenges..... 27
4.5 Retrieval Method 3: Multi-Query Generation with BERT-Based Reranking..... 27
4.5.1 Advanced Query Expansion Strategy and Architecture..... 27
4.5.2 BERT-Based Semantic Reranking Implementation..... 28
4.5.3 Document Retrieval and Deduplication Strategy..... 28
4.5.4 Advantages and Performance Characteristics..... 29
4.5.5 Technical Challenges and Limitations..... 29
4.6 Retrieval Method 4: Recursive Retrieval-Generation with Self-Correction..... 29
4.6.1 Theoretical Framework and Innovation..... 29
4.6.2 Multi-Stage Implementation Architecture..... 30
4.6.3 Knowledge Gap Analysis and Follow-up Query Generation..... 31
4.6.4 Enhanced Context Integration and Response Generation..... 31
4.6.5 Advantages and Metacognitive Capabilities..... 32
4.6.6 Technical Challenges and Limitations..... 32
4.7 Retrieval Method 5: Dynamic Prompt Engineering with Query-Adaptive Optimization..... 33
4.7.1 Adaptive Prompt Generation Framework and Philosophy..... 33
4.7.2 Query Analysis and Prompt Customization Process..... 33
4.7.3 Implementation Architecture and Process Flow..... 34
4.7.4 JSON-Based Prompt Structure and Error Handling..... 34
4.7.5 Advantages and Adaptive Capabilities..... 34
4.7.6 Limitations and Implementation Considerations..... 35
4.8 Comparative Performance Analysis and Evaluation Framework..... 35
4.8.1 Comprehensive Evaluation Methodology..... 35
4.8.2 Evaluation Dataset and Question Characteristics..... 36
4.8.3 Performance Results and Statistical Analysis..... 37
4.8.4 Statistical Significance and Confidence Analysis..... 38
4.8.5 Method Selection and Recommendations..... 39
4.9 Conclusion..... 40
Chapter 5: Production-Ready RAG Pipeline Implementation with Observability and Safety Mechanisms..... 43
5.1 Introduction..... 43
5.2 Advanced Retrieval Pipeline Architecture..... 43
5.2.1 Multi-Modal Retrieval Foundation..... 43
5.2.2 Neighbourhood Expansion for Context Preservation..... 44
5.2.3 Query Expansion with Contextual Subquery Generation..... 44
5.2.4 BERT-Based Semantic Reranking..... 44
5.2.5 Recursive Self-Correction Framework..... 45
5.3 Comprehensive Observability Through Distributed Tracing..... 45
5.3.1 Phoenix Integration and Instrumentation Architecture..... 45
5.3.2 Detailed Retrieval Step Instrumentation..... 46
5.3.3 LLM Operation Tracing and Analysis..... 46
5.3.4 End-to-End Request Correlation and Analysis..... 47
5.3.5 Debugging and Optimization Capabilities..... 47

5.4 Safety Mechanisms and Content Moderation..... **47**
5.4.1 Comprehensive Guardrail Implementation..... **47**
5.4.2 Security-Focused Query Analysis..... **48**
5.4.3 Integration with Observability Framework..... **48**
5.5 User Interface Design and Integration..... **48**
5.5.1 Gradio-Based Conversational Interface..... **48**
5.6 Performance Optimization, Implementation Advantages and Strategic Benefits..... **49**
5.6.1 Computational Efficiency Considerations..... **49**
5.6.2 Technical Architecture Advantages..... **49**
5.6.3 Operational and Maintenance Benefits..... **50**
5.6.4 Performance and Scalability Advantages..... **50**
5.7 Conclusion..... **50**
Bibliography / References..... **52**

List of Abbreviations

RAG	:	Retrieval-Augmented Generation
API	:	Application Programming Interface
OpenAPI	:	Open Application Programming Interface
OLLAMA	:	Omni-Layer Learning Language Acquisition Model
YAML	:	YAML Ain't Markup Language
JSON	:	JavaScript Object Notation
LLM	:	Large Language Model
NDCG	:	Normalized Discounted Cumulative Gain
BM25	:	Best Matching 25
CSV	:	Comma-Separated Values
HTTP	:	Hypertext Transfer Protocol
REST	:	Representational State Transfer
FAISS	:	Facebook AI Similarity Search
BGE	:	BAAI General Embedding
MMR	:	Maximum Marginal Relevance
SST	:	Similarity Score Threshold
APM	:	Application Performance Monitoring
IDF	:	Inverse Document Frequency
TF	:	Term Frequency
BERT	:	Bidirectional Encoder Representations from Transformers
UI	:	User Interface
Gradio	:	Python library for building UIs for ML model
Phoenix	:	Observability platform
GPT-4o	:	Generative Pre-trained Transformer 4 Omni

List of Tables

Table-1 : *Comparison of Retrieval + Generation Strategy*..... **41**

List of Figures

Figure-1 : System Architecture of the Retrieval-Augmented Generation (RAG)-Based..... **10**
Figure-2 : Embedding Generation Time for Each Model..... **18**
Figure-3 : Average Query Time Across Models..... **19**
Figure-4 : Average NDCG Scores for Retrieval Quality..... **19**
Figure-5 : Embedding Dimensions for Each Model..... **20**
Figure-6 : API Keyword Hit Rate Across Models..... **20**
Figure-7 : Composite Model Scores..... **21**
Figure-8 : Ensemble Retrieval Architecture Combining BM25, MMR, and SST..... **24**
Figure-9 : Neighbourhood Expansion Strategy for Context Aggregation..... **26**
Figure-10 : Multi-Query Generation with API-Aware Contextual Subqueries..... **28**
Figure-11 : Recursive Retrieval-Generation with Self-Correction Workflow..... **31**
Figure-12 : Dynamic Prompt Engineering Framework for Query-Adaptive RAG..... **33**
Figure-13 : Evaluation Framework: LLM-as-a-Judge Dimensions..... **36**
Figure-14 : Comparative Performance of Retrieval Methods (Radar Chart)..... **38**
Figure-15 : Confidence Intervals for Evaluation Metrics..... **39**
Figure-16 : Confidence Intervals for Evaluation Metrics Based on Difficulty..... **39**
Figure-17 : Correlation Matrix of Evaluation Dimensions..... **40**
Figure-18 : Production-Ready RAG Pipeline Architecture..... **43**
Figure-19 : Phoenix-Based Observability and Tracing Integration..... **46**
Figure-20 : Gradio-Based Conversational UI for API Chatbot..... **49**

Methodology

Chapter 1. System Architecture

The proposed system implements a Retrieval-Augmented Generation (RAG) pipeline specifically designed to facilitate question answering over complex API documentation. The primary objective is to enable developers to query API specifications conversationally and receive accurate, grounded responses that reduce time spent manually searching documentation.

The overall architecture comprises five key stages: Data Collection, Data Processing, Data Enrichment, Embedding Generation, and Retrieval and Generation. Each stage is designed to modularly transform raw documentation into a format suitable for retrieval-augmented question answering. The system's design is visualized in Figure 1, which provides a high-level overview of data flow and processing components.

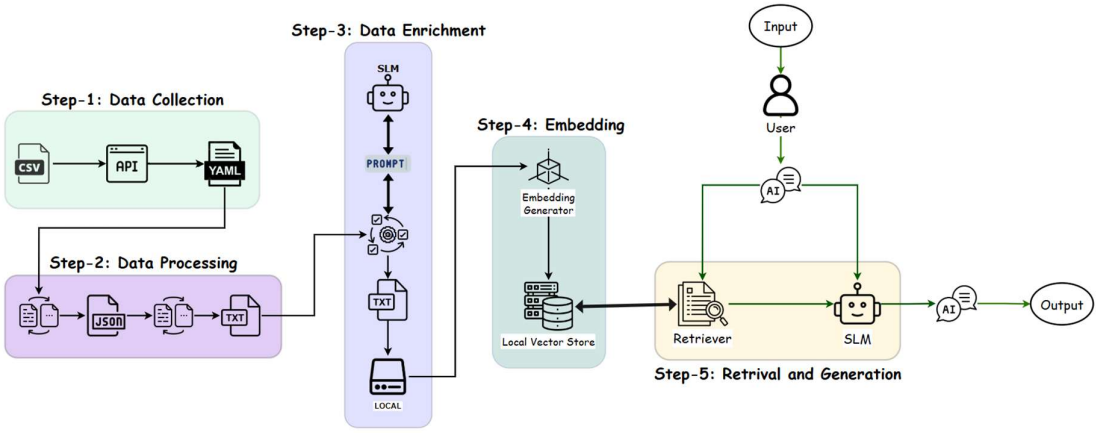


Figure 1: System Architecture of the Retrieval-Augmented Generation (RAG)-Based Chatbot – Image Link

1.1 Data Collection

The system begins with data collection from publicly available API specifications published by the organization. These specifications are typically distributed in OpenAPI YAML format and define RESTful endpoints, parameters, request/response schemas, authentication methods, and additional metadata. The primary sources include developer portals, documentation repositories, or downloadable API specification bundles.

The objective of this stage is to assemble a representative corpus of production-grade API documentation that reflects realistic complexity, heterogeneity, and scale. By collecting raw YAML files directly, the system ensures that no semantic detail is lost prior to parsing, enabling precise representation of endpoint structure and behaviour.

1.2 Data Processing

After collection, the YAML-formatted specifications are programmatically parsed into structured JSON objects to facilitate manipulation. This conversion step is critical because OpenAPI specifications can be deeply nested and require flattening for downstream text-based processing. The processing pipeline includes schema validation, error handling, and normalization to maintain consistency across diverse specifications.

Subsequently, the structured JSON data is segmented into semantically meaningful text chunks. For example, individual chunks may correspond to single endpoints, methods (GET/POST), parameter descriptions, request and response bodies, authentication requirements, or error codes. Chunking is guided by the OpenAPI schema hierarchy to ensure logical boundaries and maximize retrieval relevance.

Processed text chunks are exported to plain text files, forming an initial corpus of documentation units that can be independently retrieved at inference time. This stage enables efficient mapping between user queries and relevant API elements by providing discrete, indexable text segments.

1.3 Data Enrichment

While the raw text chunks capture the original documentation content, they may lack the semantic diversity or clarity needed for high-quality retrieval. To address this, an enrichment step is applied to augment and improve the text data.

Prompt-based generation using a large language model (LLM) is applied to rephrase, expand, or clarify certain text sections. For example, complex parameter descriptions may be paraphrased into simpler language, or additional context may be generated to improve semantic matching during retrieval. This process improves the model's ability to understand diverse user query formulations while preserving technical accuracy.

Enriched text data is stored locally in a structured repository. Maintaining local control over the data ensures security, reproducibility, and ease of iteration during development.

1.4 Embedding Generation

The enriched text corpus is then transformed into dense vector representations using an embedding model. The embedding generator used is selected depending on performance requirements and cost constraints. The goal is to encode the semantic meaning of each text chunk into a high-dimensional vector that enables best similarity-based retrieval.

Embedding generation is a critical step, as retrieval effectiveness relies on the quality of semantic representation. During development, multiple embedding models were compared on criteria such as semantic relevance, latency, and memory footprint. The selected embedding model produces embeddings that are stored in a local vector database.

1.5 Retrieval and Generation

At inference time, the system supports interactive question answering through a console-based interface. The user enters a natural language query, which is first embedded using the same embedding model. The retriever module performs a vector similarity search against the local store to identify the most relevant documentation chunks based on semantic proximity.

The retrieved context is then provided as input to the generation module, which uses a large language model (LLM) to produce a natural language response. The generation process is designed to be retrieval-augmented, ensuring that the answer remains grounded in retrieved documentation and minimizes unsupported hallucinations.

The UI interface serves as a lightweight demonstration environment to validate the integrated pipeline. It allows developers to test realistic queries and observe retrieved evidence and generated responses in real time. This prototype approach enables iterative evaluation and refinement of retrieval strategies, embedding models, and generation prompts.

1.6 Design Considerations and Modularity

The architecture is designed to be modular and extensible. Each component data processing, enrichment, embedding, retrieval, generation can be independently updated or replaced without requiring reimplementation of the entire pipeline. This separation of concerns supports experimentation with alternative embedding models, retrievers (e.g., BM25 vs. dense retrieval), or generative backends.

The modular design also enables future scaling, such as deployment via web interfaces, support for multiple API domains, or integration with additional developer tools. By grounding generation in retrieved, organization-authored specifications, the system provides a foundation for trustworthy, transparent, and efficient conversational access to API documentation.

Chapter 2. Data Collection and Processing

The first critical phase in developing the Retrieval-Augmented Generation (RAG)-based chatbot is the acquisition and transformation of high-quality API documentation into a retrieval-friendly corpus. The system targets publicly available OpenAPI specifications that define the organization's REST APIs. These structured documents, typically authored in YAML format, capture detailed information about endpoints, parameters, request/response schemas, authentication requirements, and associated metadata.

2.1 Data Collection

The data collection workflow begins with a curated CSV file that enumerates categories of APIs along with the corresponding URLs where their OpenAPI YAML specifications can be obtained. This approach ensures coverage of multiple API domains, allowing the chatbot to be tested against a realistic, heterogeneous documentation set.

For each entry in the CSV, the system automatically retrieves the OpenAPI YAML specification via HTTP requests. Downloaded specifications are stored locally in a structured directory hierarchy, preserving their original formatting. This local archival guaranteed reproducibility of experiments and supports version control of the raw input data.

A significant design choice in this phase is to retain the original YAML format until explicitly converted, avoiding any lossy preprocessing. By ensuring that specifications are collected in their source-authorized form, semantic and structural fidelity is maintained, supporting downstream parsing without loss of detail.

2.2 Data Processing and Format Conversion

Once collected, the YAML specifications are programmatically parsed and converted to JSON format. This transformation is crucial because JSON representations are more readily manipulated in most programming environments and allow easy traversal of nested hierarchical structures. Each converted JSON file maintains the complete OpenAPI schema, including paths, methods, parameters, security schemes, and reusable components.

The processing pipeline applies schema validation and error handling during conversion to identify and filter out invalid or incomplete specifications. Successfully converted JSON files are stored locally in a dedicated directory structure, serving as the canonical structured input for further analysis.

2.3 Specification Parsing and Chunking

The central goal of data processing is to transform large, complex API specifications into semantically meaningful text units that can be efficiently indexed and retrieved. To achieve this, the system parses each OpenAPI JSON document to extract detailed information about every defined endpoint. This includes:

- Endpoint paths and HTTP methods (e.g., GET, POST)
- Summaries and detailed descriptions
- Tags and categorization metadata
- Parameters, including names, locations, required/optional status, and descriptions

- Request body schemas and example JSON structures
    - Response schemas, status codes, and example responses
    - Security requirements, such as authentication schemes
    - Custom extension fields prefixed with "x-" in the OpenAPI standard

This information is programmatically extracted and structured into discrete text chunks, each representing a single endpoint and its properties in a consistent, human-readable format. The design emphasizes faithful representation of the original documentation while reformatting it to support retrieval by semantic similarity.

2.4 RAG-Optimized Formatting

In addition to direct extraction, the processing pipeline converts the text with several formatting steps aimed at improving its suitability for retrieval-augmented generation. Schema definitions are traversed recursively to list nested properties, data types, and associated descriptions. Sample JSON objects are programmatically generated from schema definitions to provide concrete, example payloads for both requests and responses.

The final output for each endpoint is a structured, detailed text document organized into clearly labelled sections such as ENDPOINT, PATH, METHOD, TAGS, DESCRIPTION, PARAMETERS, REQUEST BODY, RESPONSES, and SECURITY. These consistent structural markers enable fine-grained chunking and improve semantic retrieval precision.

2.5 Data Enrichment

Despite careful data collection and processing, the extracted endpoint documentation remains limited in depth and semantic diversity. Typical OpenAPI specifications, even when cleanly parsed and structured, often contain minimal natural-language context, focusing narrowly on schema details, parameter names, and terse descriptions. Such content is insufficient for effective retrieval-augmented generation (RAG) applications, where semantic similarity search thrives on rich, varied, and human-readable text.

To address this limitation, a dedicated Data Enrichment step was designed and implemented. This step leverages a Large Language Model (LLM) to automatically transform the raw processed endpoint documentation into highly structured, retrieval-optimized text files. The enrichment pipeline systematically expands each endpoint's sparse technical description into a semantically rich, multi-section document designed to support both retrieval and generation tasks.

2.5.1 Motivation for Enrichment

The primary motivation is to bridge the gap between structured API reference data and natural-language information needs. While developers or end-users often ask questions in free-form language (e.g., "How do I delete a user profile?"), the raw documentation might only specify DELETE /user/{id} with no further context. Without enrichment, vector retrieval models struggle to match such queries to the right chunk, resulting in poor RAG performance.

Enrichment addresses this by introducing:

- **Natural-language overviews** summarizing endpoint purpose and usage.
- **Relevant search terms** to improve embedding space coverage.
- **Example user questions** that mimic real query intent.

- **Developer-focused notes** highlighting parameters, request/response schemas, and security considerations.
- **Consistently formatted raw documentation** for full fidelity.

These elements collectively ensure that retrieval is grounded not just in keyword overlap, but in semantic similarity informed by realistic usage contexts.

2.5.2 Automated Enrichment Pipeline

The enrichment pipeline is designed as a batch process operating over the corpus of processed endpoint text files generated in the earlier step. The key stages of this pipeline are

Input Preparation

- All processed text files are collected from a dedicated directory.
- Each file typically contains multiple endpoint sections, separated by well-defined delimiters.
- These sections are split to ensure that enrichment is performed at the *endpoint* level granularity, preserving chunk-level retrieval fidelity.

Prompt Engineering

- A carefully designed enrichment prompt is formulated to guide the LLM in transforming raw endpoint text.
- The prompt includes detailed instructions to produce consistent output with markdown headings, bullet lists, and code blocks where appropriate.
- Importantly, the prompt explicitly instructs the model to use only content from the raw documentation when extracting details, while allowing generic additions for search terms and example questions to boost retrieval effectiveness.

LLM Invocation

- For each endpoint chunk, the system constructs a complete prompt by embedding the raw documentation into the template.
- This prompt is then sent to a locally hosted Large Language Model (LLM), chosen for its balance of performance, controllability, and cost.
- The LLM's deterministic temperature setting ensures consistent, reproducible results across runs.

2.6 Structured Markdown Generation

The model responds with a markdown-formatted document containing the following sections:

- A concise natural-language summary of the endpoint's purpose, HTTP method, path, and security requirements.
- 5–10 keywords or phrases relevant for semantic search.
- Sample natural-language queries likely to match the endpoint.
- Technical details about parameters, request/response schemas, required fields, and error handling.
- Cleaned and consistently formatted reproduction of the original processed text.

This format is designed to optimize both retrieval accuracy (via search terms and user questions) and grounding fidelity (via developer notes and raw documentation). Each enriched endpoint is saved as an individual markdown-formatted text file.

2.7 Advantages of the Enrichment Approach

- **Improved Semantic Retrieval**: By expanding sparse documentation into rich natural language, embedding models can better learn meaningful similarities between user queries and document chunks.
- **Grounded Context**: The structured prompt ensures that enrichment remains faithful to source content, avoiding hallucination while adding helpful context.
- **Consistency Across Endpoints**: Standardized markdown sections enable reliable chunking, indexing, and retrieval.
- **Ease of Maintenance**: New or updated API specifications can be reprocessed and re-enriched automatically, ensuring scalability.

Chapter 3. Embedding Comparison and Creation

After enriching the raw API documentation with structured, retrieval-friendly content, the next critical step in the Retrieval-Augmented Generation (RAG) pipeline is generating high-quality vector embeddings. The goal of this stage is to convert enriched text chunks into numerical representations that preserve semantic similarity for effective retrieval. To ensure optimal performance, multiple embedding models were evaluated and compared across several key metrics, including embedding generation time, query response time, and retrieval quality.

3.1 Motivation and Goals

Embedding quality fundamentally determines the effectiveness of the entire RAG system. If embeddings fail to capture the semantic meaning of user queries and documentation chunks, the retrieval component will return irrelevant or suboptimal results, directly harming the end-user experience. Moreover, practical considerations such as computational cost (embedding time) and serving latency (query time) are also essential, especially for real-time applications.

Given these requirements, the evaluation aimed to identify and embedding model that offers a good balance between:

- Retrieval quality (measured by NDCG score on realistic queries),
- Embedding time (the one-time cost of generating the vector store),
- Query time (the per-request retrieval latency),
- Embedding dimension (affecting storage and compute costs).

3.2 Experimental Setup

The enriched corpus was first split into context-preserving chunks using overlap-aware splitting. Each chunk retained consistent metadata to support traceability.

Three embedding models with varying architectures and dimensionalities were then configured:

- A lightweight Hugging Face model (384 dimensions),
- Nomic Embed (768 dimensions),
- BGE-M3 (1024 dimensions).

These models were selected to represent a spectrum of trade-offs in quality, dimension size, and compute requirements.

3.2.1 Evaluation Process

The evaluation was designed to mimic realistic retrieval scenarios for API documentation chatbots. It followed these main stages:

Vector Store Creation

- Each model generated embeddings for all text chunks and stored them in a FAISS index.
- Embedding generation time was recorded as an important cost metric.

Query Performance Measurement

- A curated set of real-world API-related test queries was used to simulate chatbot questions.
- For each model, the average retrieval latency (query time) was measured.

Relevance Scoring and Ranking Evaluation

- Retrieved chunks were scored for relevance using a heuristic combining keyword overlap and presence of API-specific terms.
- Normalized Discounted Cumulative Gain (NDCG) was computed to measure the quality of ranking, reflecting how well each model prioritized relevant results.

API-Specific Keyword Coverage

- An analysis was conducted to measure each model's ability to retrieve text containing key API-related concepts (e.g., endpoint, parameters, security, authentication).
- This metric assessed domain grounding beyond generic semantic similarity.

Composite Scoring and Recommendation

The models were compared on normalized, weighted metrics to identify the best overall choice for production deployment.

3.3 Results and Interpretation

3.3.1 Embedding Time Comparison

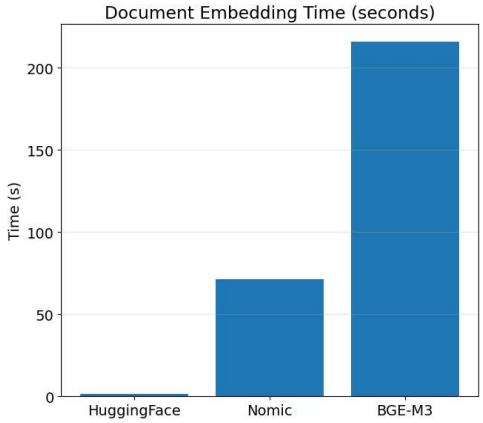


Figure-2: Embedding Generation Time for Each Model

A bar plot comparing embedding generation times showed large variations between models. The Hugging Face model completed embedding in just over ~1.6 seconds, while Nomic Embed required over ~70 seconds, and BGE-M3 exceeded ~215 seconds. This indicates a clear cost advantage for Hugging Face in workflows where embeddings are regenerated frequently (e.g., for dynamic or frequently updated documentation).

3.3.2 Query Time Analysis

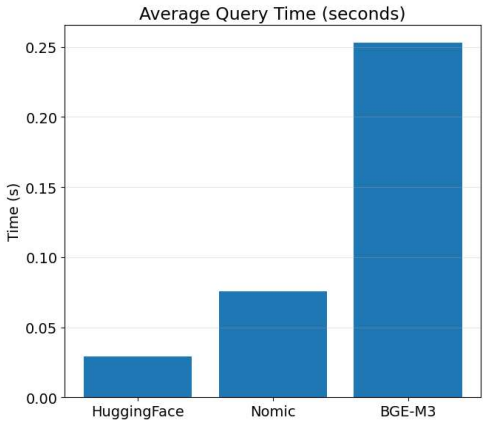


Figure-3: Average Query Time Across Models

The average retrieval latency per query was lowest for the Hugging Face model, with sub-0.05 sec performance. In contrast, Nomic Embed and BGE-M3 showed higher latencies. Low query times are particularly important for conversational AI applications, where users expect near-instant responses.

3.3.3 Retrieval Quality (NDCG)

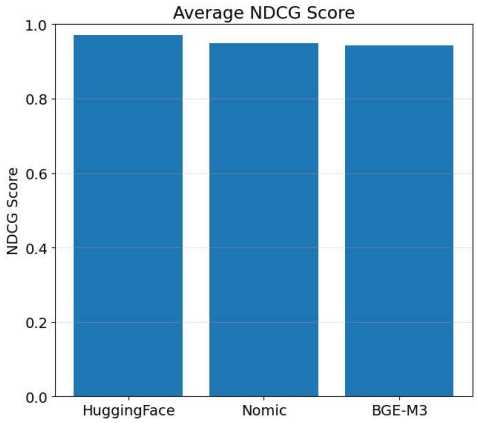


Figure-4: Average NDCG Scores for Retrieval Quality

All models achieved high average NDCG scores, reflecting effective semantic retrieval of API documentation chunks. These results indicate that even the smallest model achieved strong ranking quality, challenging assumptions that larger embeddings automatically yield better retrieval.

3.3.4 Embedding Dimension Considerations

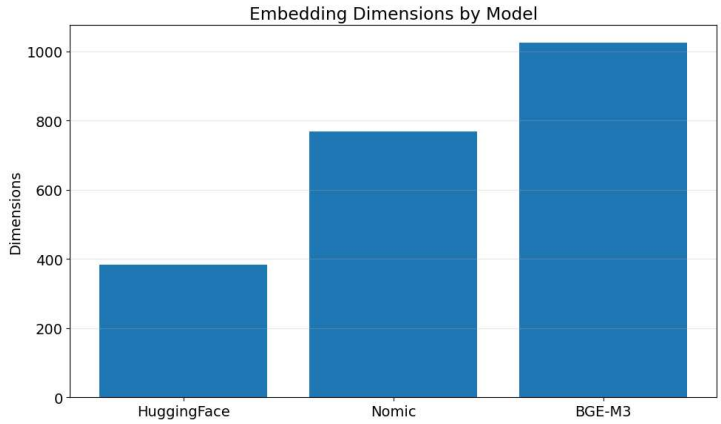


Figure-5: Embedding Dimensions for Each Model

Embedding dimensions directly impact index size, storage requirements, and retrieval latency. Hugging Face's 384-dimensional embeddings offer a significant efficiency advantage, while BGE-M3's 1024-dimensional vectors demand greater resources for comparable quality.

3.3.5 API Keyword Coverage

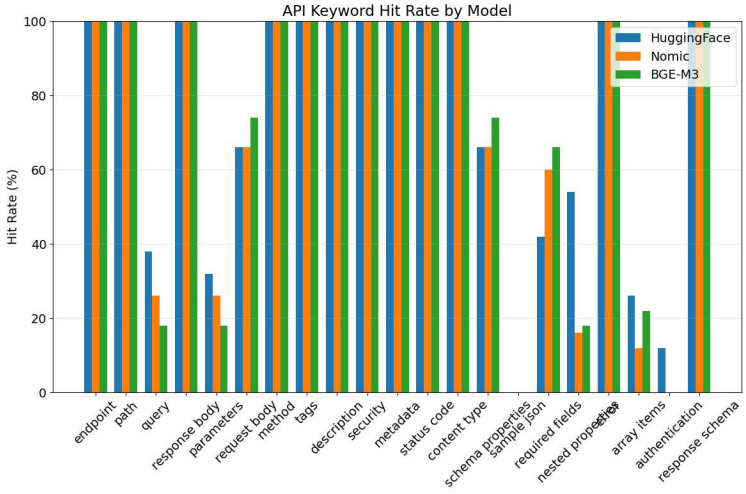


Figure-6: API Keyword Hit Rate Across Models

An additional analysis focused on the proportion of retrieved text chunks containing essential API concepts. Across all models, coverage of terms such as *endpoint*, *path*, *parameters*, and *authentication* was high, confirming that all embeddings maintained strong domain relevance.

However, slight variations were observed, with larger models sometimes surfacing more context-rich passages at the cost of latency.

3.3.6 Composite Score and Model Recommendation

A final composite score was computed for each model, weighting:

- Retrieval quality (NDCG): highest importance,
- Query time: medium importance,
- Embedding time: lower importance (as a one-time cost).

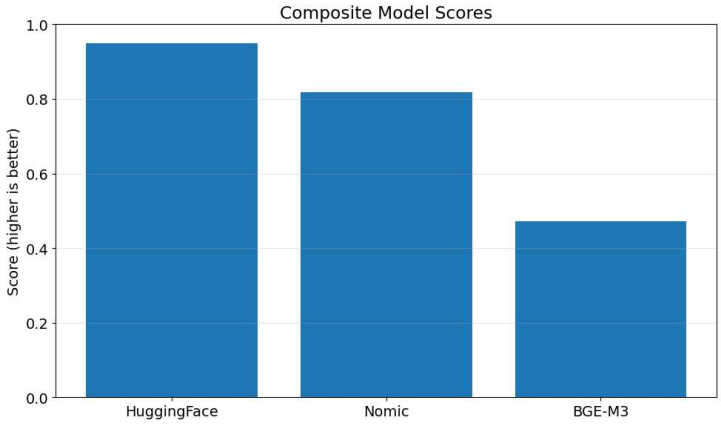


Figure-7: Composite Model Scores

The analysis concluded that the **Hugging Face** model provided the best balance between retrieval accuracy, query responsiveness, and computational cost.

Its combination of minimal embedding time, fastest query time, and highest NDCG score makes it the recommended choice for production deployment in this API-focused RAG chatbot.

3.4 Saving and Deployment

Following selection, the best-performing embedding model was used to create a finalized FAISS vector store of the enriched documentation. This vector index was saved locally for seamless integration into the chatbot's retrieval pipeline.

Chunks were saved with overlap to preserve context, and metadata ensured traceability to the source files. This design enables efficient, high-quality retrieval during chatbot operation while allowing easy re-embedding in the future as documentation evolves.

Chapter 4. Comparative Analysis and Selection of Retrieval Strategies

4.1 Introduction

The effectiveness of a Retrieval-Augmented Generation (RAG) system fundamentally depends on the quality and relevance of the retrieved documents that provide context to the language model. While traditional RAG implementations often rely on simple similarity-based retrieval, this research explores advanced retrieval strategies that can significantly enhance the system's performance across different query types and complexities. This chapter presents a comprehensive comparative analysis of five distinct retrieval methodologies implemented and evaluated within the CyberArk API documentation context.

The retrieval strategies examined range from ensemble-based approaches that combine multiple retrieval methods to sophisticated techniques involving query expansion, neighbourhood context inclusion, and dynamic prompt engineering. Each method addresses specific limitations inherent in basic vector similarity searches, such as semantic gaps, context fragmentation, and query ambiguity. The comparative analysis employs a rigorous evaluation framework using an LLM-as-a-judge methodology across six key performance dimensions: accuracy, completeness, conciseness, clarity and readability, adherence to instructions, and relevance to expected answers.

The study utilizes the OpenAI GPT-4o-mini model as the language generation component and implements comprehensive rate limiting with a maximum of 0.1 requests per second to ensure stable performance. The evaluation dataset comprises diverse question types across multiple difficulty levels and API documentation categories, providing a robust foundation for assessing retrieval method effectiveness.

4.2 Baseline Infrastructure and Configuration

4.2.1 Embedding Model and Vector Store Architecture

The foundation of all retrieval methods implemented in this study relies on the HuggingFace - all-MiniLM-L6-v2 embedding model, which produces 384-dimensional dense vectors that capture semantic relationships between text segments. This sentence transformer model was selected for its optimal balance between computational efficiency and semantic representation quality, having demonstrated proven performance on semantic textual similarity tasks while maintaining a relatively compact model size suitable for production deployment.

The vector store implementation utilizes Facebook AI Similarity Search (FAISS) as the underlying database, providing efficient approximate nearest neighbour search capabilities essential for large-scale document retrieval. The FAISS implementation enables multiple search strategies including standard similarity search, Maximum Marginal Relevance (MMR), and similarity score thresholding, forming the foundation for the various retrieval methods examined in this study.

The document preprocessing pipeline involves sophisticated chunking strategies that balance context preservation with retrieval granularity. The chunks are enriched through ChatGPT-based enhancement processes that improve the semantic density and contextual clarity of the stored content. This preprocessing approach ensures that the retrieved context maintains both local coherence and global document structure awareness.

4.2.2 System Prompt Architecture and Configuration

A critical component of the RAG system is the comprehensive system prompt that provides consistent behavioural guidelines across all retrieval methods. The system prompt establishes strict context reliance principles, requiring the model to answer exclusively based on provided documentation context while explicitly prohibiting hallucination or external knowledge injection. The prompt incorporates specific response behaviours for different query types, including greeting handling, API endpoint specifications, and general functionality explanations.

The prompt architecture includes detailed formatting guidelines requiring Markdown structuring for enhanced readability, comprehensive endpoint documentation standards including HTTP methods, parameters, security requirements, and schema specifications. Error handling protocols are embedded within the prompt to manage scenarios where requested information is not available in the provided context, ensuring transparent communication of system limitations.

4.3 Retrieval Method 1: Ensemble Retrieval Search

4.3.1 Conceptual Framework and Implementation

The ensemble retrieval method represents a sophisticated approach that combines three distinct retrieval strategies to leverage their complementary strengths. This method addresses the fundamental limitation that no single retrieval strategy performs optimally across all query types and contexts, implementing a weighted voting mechanism to balance different retrieval paradigms.

The ensemble architecture incorporates three component retrievers with carefully calibrated weights: a BM25 retriever (40% weight) providing keyword-based retrieval excellence for specific technical terms and exact matches; a Maximum Marginal Relevance (MMR) retriever (30% weight) utilizing vector similarity search with diversity optimization to prevent redundant document selection; and a Similarity Score Threshold (SST) retriever (30% weight) implementing vector similarity search with a quality threshold of 0.3 to ensure only sufficiently relevant documents are included.

The weight distribution reflects empirical optimization based on the characteristics of technical documentation retrieval, where exact keyword matching often provides the highest precision for API-specific queries, while semantic methods contribute valuable context and related information. The ensemble implementation includes robust document extraction mechanisms that handle different FAISS docstore structures gracefully, ensuring compatibility across different system configurations.

4.3.2 Technical Implementation Details

The ensemble method's implementation demonstrates sophisticated error handling and compatibility management. The document extraction process implements a try-catch mechanism that first attempts to access documents through the primary docstore interface, falling back to alternative access methods when encountering different FAISS version configurations. This approach ensures system reliability across diverse deployment environments.

The BM25 component utilizes the rank_bm25 implementation with careful document preprocessing to optimize keyword matching effectiveness. The configuration sets k=2 for each component retriever, resulting in a potential maximum of six documents before deduplication, though

the actual number varies based on overlap between different retrieval methods. The MMR component employs FAISS's built-in MMR functionality with optimized parameters to balance relevance and diversity in the retrieved document set.

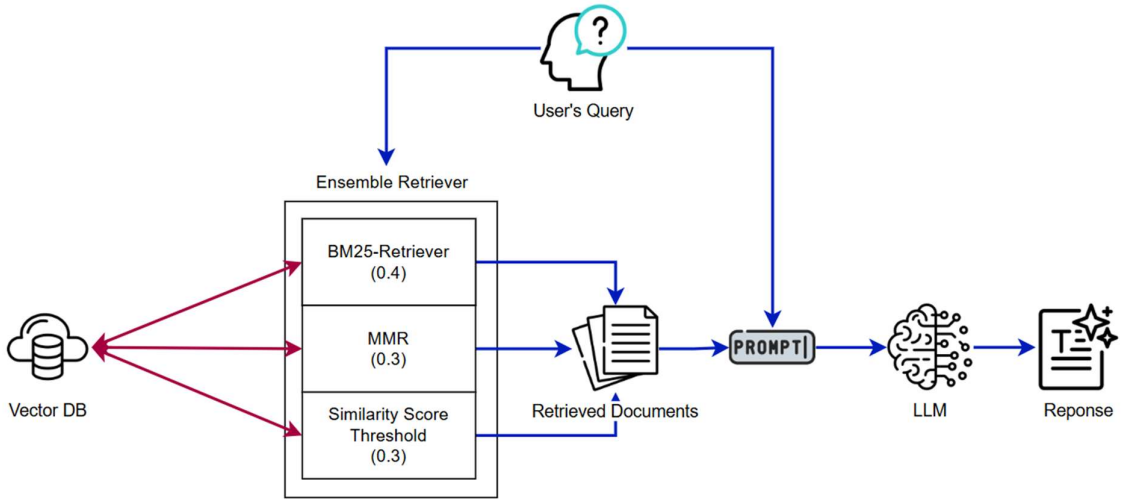


Figure-8: Ensemble Retrieval Architecture Combining BM25, MMR, and SST

4.3.3 Advantages and Performance Characteristics

The ensemble approach provides several significant advantages in technical documentation retrieval scenarios. The complementary coverage achieved through combining lexical and semantic retrieval ensures comprehensive handling of both keyword-specific queries and conceptually similar information needs. This balanced approach provides consistently good performance across diverse query types without extreme optimization for specific scenarios, making it suitable for production environments with varied user requirements.

The method demonstrates strength in handling ambiguous queries where different retrieval paradigms might identify different relevant aspects of the information need. The weighted combination allows the system to leverage the precision of keyword matching while benefiting from the recall capabilities of semantic search, resulting in robust performance across the evaluation dataset.

4.3.4 Limitations and Considerations

Despite its strengths, the ensemble method exhibits certain limitations that impact its effectiveness in specific scenarios. The static weight distribution (40%-30%-30%) represents a fixed compromise that may not be optimal for all query types, potentially underperforming in scenarios where dynamic weight adjustment based on query characteristics would be beneficial. The computational overhead of running three separate retrieval processes increases processing time and resource requirements compared to single-method approaches.

The method's inability to dynamically adapt its strategy based on query characteristics or complexity represents a significant limitation in scenarios requiring specialized retrieval approaches. Additionally, the fixed threshold values and parameters may require domain-specific tuning for optimal performance in different technical documentation contexts.

4.4 Retrieval Method 2: Neighbourhood Expansion with Context Aggregation

4.4.1 Theoretical Foundation and Motivation

The neighbourhood expansion method addresses a critical limitation in traditional chunk-based retrieval systems: the fragmentation of contextual information across adjacent document segments. This approach recognizes that technical documentation often contains information that spans multiple consecutive chunks, and retrieving isolated fragments may provide incomplete or misleading context to the language model. The method implements a sophisticated understanding of document structure and chunk relationships to reconstruct coherent contextual narratives.

The theoretical foundation rests on the principle that document authors structure information in logical sequences, where understanding of a particular concept often requires access to preceding explanations, examples, or subsequent elaborations. In API documentation specifically, endpoint descriptions frequently include prerequisite information, detailed parameter explanations, and example implementations that may be distributed across multiple chunks but are essential for complete understanding.

4.4.2 Implementation Architecture and Process Flow

The neighbourhood expansion method implements a sophisticated two-stage retrieval process designed to maximize context completeness while maintaining relevance. The first stage performs parallel retrieval using both similarity-based and BM25 methods, implementing a common document identification algorithm that enhances confidence in document relevance through cross-method validation. This approach leverages the strengths of both lexical and semantic matching to identify the most promising document segments.

The common document identification function utilizes metadata-based matching on document index and chunk index parameters, creating a robust mechanism for identifying overlapping results between different retrieval methods. When common documents are identified, the system prioritizes these high confidence matches; when no overlap exists, the method gracefully falls back to selecting top documents from each retrieval approach, ensuring consistent output regardless of query characteristics.

The second stage implements the core neighbourhood expansion functionality, retrieving adjacent chunks within the same document to provide comprehensive contextual information. The system retrieves up to two chunks in each direction (n=2) from the identified relevant chunks, resulting in context windows that can span up to five consecutive chunks per identified document. This expansion respects document boundaries to prevent contamination between different API documentation sections while maximizing relevant context within individual documents.

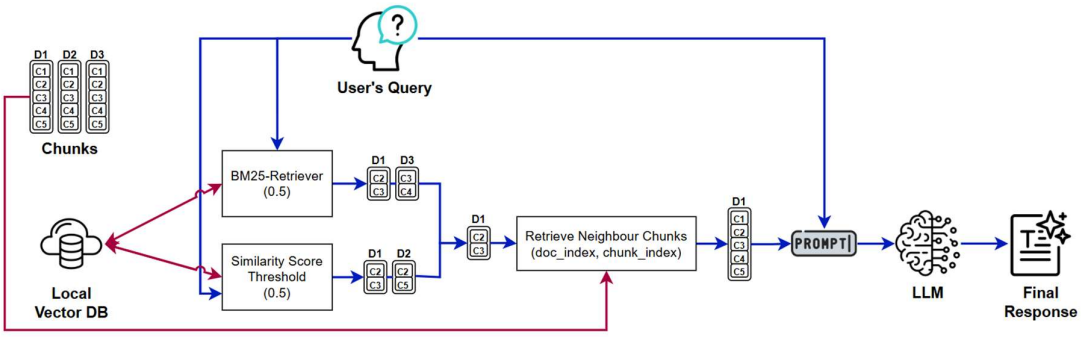


Figure-9: Neighbourhood Expansion Strategy for Context Aggregation

4.4.3 Context Window Management and Optimization

The implementation includes sophisticated context window management that balances comprehensiveness with computational efficiency. The adjacent document retrieval function sorts chunks by their position within the document using metadata-based indexing, ensuring that the expanded context maintains logical flow and coherence. The system calculates previous and next chunk boundaries dynamically, handling edge cases where the target chunk is near document beginnings or endings.

The method incorporates intelligent boundary detection that prevents context expansion beyond document limits, ensuring that each expanded context window represents a coherent segment of the original documentation structure. Debug information provides detailed insights into the expansion process, including the number of chunks retrieved, document names, and position information, facilitating system optimization and troubleshooting.

4.4.4 Advantages and Performance Benefits

The neighbourhood expansion method provides significant advantages in scenarios requiring comprehensive contextual understanding. Enhanced context preservation ensures that complete explanations, step-by-step procedures, and related concepts are maintained together, substantially reducing context fragmentation issues that plague simpler retrieval methods. This approach is particularly beneficial for complex API documentation where understanding often requires access to multiple related concepts presented in logical sequence.

The method demonstrates superior performance on multi-step queries and procedural questions where the complete context is essential for accurate response generation. The intelligent boundary handling prevents contamination between different documentation sections while maximizing relevant context within documents, resulting in higher-quality retrieved content that better supports accurate response generation.

The metadata-driven organization maintains logical document structure throughout the expansion process, ensuring that the expanded context preserves the author's intended information flow and conceptual relationships. This structural awareness significantly improves the quality of context provided to the language model, resulting in more accurate and complete responses.

4.4.5 Limitations and Technical Challenges

The neighbourhood expansion method faces several important limitations that impact its applicability in certain scenarios. The significant increase in context window usage due to expansion can approach language model context limits, particularly when multiple documents are retrieved and expanded. This increased token count also results in higher processing costs and longer response times, which may be problematic in cost-sensitive or time-critical applications.

The potential for noise introduction represents another significant challenge, as adjacent chunks may contain tangentially related information that could dilute the relevance of the core retrieved content. This noise can potentially confuse the language model and reduce response quality, particularly in cases where the expanded context includes information that is structurally adjacent but conceptually unrelated to the user's query.

The computational complexity of the two-stage retrieval process and subsequent neighbourhood expansion increases system overhead compared to simpler retrieval methods. The method requires careful parameter tuning to balance context completeness with computational efficiency, and the optimal expansion parameters (n=2) may not be suitable for all document types or query characteristics.

4.5 Retrieval Method 3: Multi-Query Generation with BERT-Based Reranking

4.5.1 Advanced Query Expansion Strategy and Architecture

The multi-query generation with BERT-based reranking method implements a sophisticated query expansion approach that addresses inherent ambiguity and specificity challenges in natural language queries. This method recognizes that user queries often contain implicit information needs and may benefit from multiple interpretations to achieve comprehensive information retrieval. The core innovation lies in leveraging the language model itself to generate multiple refined subqueries that capture different aspects and interpretations of the user's information need.

The query expansion strategy begins with contextual analysis using keyword matching against a curated knowledge base of API endpoint descriptions. This initial analysis provides domain-specific context that informs the subsequent query generation process, ensuring that expanded queries remain within the bounds of available documentation and API capabilities. The system loads and processes one-liner descriptions organized by category, creating a structured knowledge base that guides the query expansion process.

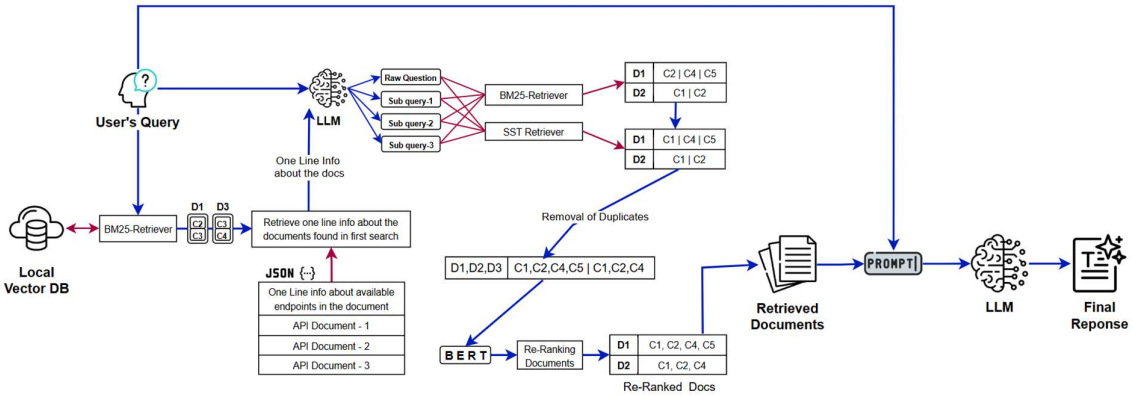


Figure-10: Multi-Query Generation with API-Aware Contextual Subqueries

4.5.2 BERT-Based Semantic Reranking Implementation

The reranking component implements semantic similarity scoring using the same sentence transformer model employed for the vector store, ensuring consistency in semantic representation across the retrieval pipeline. This consistency is crucial for maintaining coherent similarity calculations and preventing artifacts that might arise from using different embedding models for different components of the retrieval process.

The reranking process begins with document retrieval using the expanded query set, followed by comprehensive deduplication based on document and chunk indices. The system then computes BERT embeddings for both the original query and all retrieved document contents, calculating cosine similarity scores between the query embedding and each document embedding. This semantic scoring provides a more nuanced relevance assessment than simple retrieval frequency or basic similarity measures.

The implementation includes intelligent filtering using a similarity threshold of 0.2, which removes documents that fall below minimum relevance standards while preserving documents that demonstrate meaningful semantic relationships to the query. The threshold selection represents a balance between precision and recall, ensuring that marginally relevant documents are excluded while maintaining sufficient context for comprehensive response generation.

4.5.3 Document Retrieval and Deduplication Strategy

The multi-query approach necessarily generates multiple documents sets from different subqueries, requiring sophisticated deduplication strategies to prevent redundant context and optimize computational efficiency. The system implements metadata-based deduplication using document index and chunk index combinations, creating unique identifiers that reliably identify duplicate content across different retrieval operations.

The deduplication process includes robust error handling for documents with missing or incomplete metadata, ensuring system stability while maximizing the utility of available content. The implementation tracks the deduplication process for debugging and optimization purposes, providing detailed statistics on document retrieval and filtering effectiveness. The final document set represents a comprehensive but non-redundant collection of content relevant to the various interpretations of the user's query. This approach maximizes information coverage while

maintaining computational efficiency and preventing context pollution through duplicate content inclusion.

4.5.4 Advantages and Performance Characteristics

The multi-query generation with BERT-based reranking method provides several significant advantages in handling complex and ambiguous information needs. Query ambiguity resolution represents a key strength, as the method effectively handles uncertain or under-specified queries by exploring multiple interpretations and aspects of the information need. This capability is particularly valuable in technical documentation contexts where users may not be familiar with precise terminology or may be exploring related concepts.

The semantic quality control implemented through BERT-based reranking with similarity thresholding ensures that only semantically relevant documents are included in the final context. This filtering significantly improves the signal-to-noise ratio of retrieved content, reducing the likelihood of irrelevant information confusing the language model or degrading response quality. The API-aware context generation represents another significant advantage, as the integration of domain-specific knowledge during query generation ensures that expanded queries remain within the scope of available documentation. This approach prevents the generation of queries that might be semantically valid but outside the domain of the available knowledge base.

4.5.5 Technical Challenges and Limitations

Despite its sophisticated approach, the multi-query generation method faces several important limitations that impact its practical implementation. The dependency on additional LLM calls for query generation increases both latency and computational costs, potentially making the method unsuitable for applications with strict performance requirements or limited computational budgets.

The similarity threshold of 0.2 represents a critical parameter that significantly impacts system performance, but the optimal value may vary based on domain characteristics, document types, and embedding model properties. This threshold sensitivity requires careful tuning and potentially adaptive adjustment mechanisms to maintain optimal performance across different scenarios. The complexity of the multi-stage process, involving query expansion, retrieval, and reranking, makes it challenging to diagnose and optimize individual components. Performance issues or quality problems may arise from any stage of the pipeline, and isolating the source of problems requires comprehensive monitoring and debugging capabilities.

The method's reliance on the language model's ability to generate appropriate subqueries represents a potential point of failure, as poor query generation can lead to irrelevant retrieval results that may not be adequately filtered by the reranking process. The quality of the overall system is therefore dependent on the consistency and accuracy of the query generation component.

4.6 Retrieval Method 4: Recursive Retrieval-Generation with Self-Correction

4.6.1 Theoretical Framework and Innovation

The recursive retrieval-generation method represents the most sophisticated approach examined in this study, implementing an iterative improvement cycle that mimics human information-

seeking behaviour. This method addresses the fundamental limitation that initial retrieval attempts may not capture all necessary information to provide complete and accurate responses, particularly for complex queries that require information synthesis from multiple sources or perspectives. The theoretical foundation rests on the concept of metacognitive information processing, where the system develops the capability to assess its own knowledge state and identify areas requiring additional information. This approach recognizes that high-quality response generation often requires iterative refinement, where initial responses reveal knowledge gaps that can be systematically addressed through targeted additional retrieval.

The method implements a form of computational reflection, enabling the system to evaluate its initial output critically and formulate specific follow-up queries designed to address identified deficiencies. This recursive approach allows the system to converge toward higher-quality responses through systematic gap identification and targeted information gathering.

4.6.2 Multi-Stage Implementation Architecture

The recursive retrieval-generation method implements a sophisticated five-stage process designed to maximize response quality through iterative improvement. The first stage performs initial retrieval using the previously validated BERT-based reranking approach, providing a high-quality foundation for initial response generation. This stage leverages the proven effectiveness of semantic reranking while establishing the baseline context for subsequent processing.

The second stage generates an initial response using the standard system prompt and retrieved context, creating a preliminary answer that serves as the foundation for gap analysis. This initial response represents the system's best attempt at answering the query using the initially retrieved information, providing a concrete artifact for subsequent evaluation and improvement.

The third stage implements the core innovation of knowledge gap analysis, where the system employs the language model as a critical evaluator to identify areas where the initial response is incomplete, ambiguous, or lacking in necessary detail. The gap identification prompt specifically directs the model to consider missing technical details, ambiguous statements requiring clarification, incompletely explained API parameters or behaviours, missing examples or use cases, and areas where the response expresses uncertainty.

The fourth stage uses the follow up queries and call the neighbourhood expansion method to get the neighbouring documents which can help in improvement of the context.

The fifth stage is where we use de-duplication techniques to remove duplicate from the documents, and we combine the neighbourhood retrieved documents and initially retrieved documents and pass it to LLM to generate response.

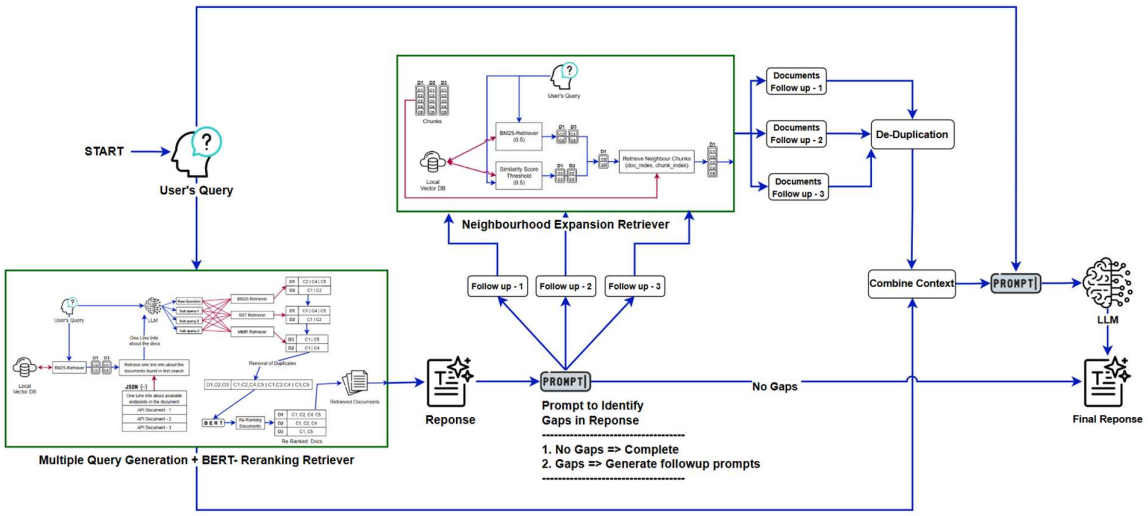


Figure-11: Recursive Retrieval-Generation with Self-Correction Workflow

4.6.3 Knowledge Gap Analysis and Follow-up Query Generation

The knowledge gap analysis component represents a sophisticated implementation of automated response evaluation and improvement planning. The gap identification prompt provides specific guidance on the types of deficiencies to identify, ensuring comprehensive coverage of potential response quality issues. The prompt specifically targets technical details that would improve completeness, ambiguous statements needing clarification, API parameters or behaviours requiring fuller explanation, missing examples that would enhance understanding, and explicit acknowledgments of uncertainty that suggest additional information needs.

The gap analysis process generates specific follow-up queries formulated to address identified deficiencies through targeted retrieval. These follow-up queries are designed to be actionable and specific, enabling effective retrieval of supplementary information that directly addresses the identified gaps. The system implements intelligent gap assessment, returning a "COMPLETE" indicator when no significant gaps are identified, preventing unnecessary additional processing when the initial response is deemed sufficient. The follow-up query generation process demonstrates the system's ability to perform targeted information seeking based on self-assessment of response quality. This capability represents a significant advancement over static retrieval methods, enabling dynamic adaptation to the specific information needs revealed through initial response generation.

4.6.4 Enhanced Context Integration and Response Generation

The fourth stage implements sophisticated additional context retrieval using different methods to ensure diversity in the supplementary information. The system uses neighbourhood expansion retrieval for follow-up queries, leveraging the strengths of context aggregation to gather comprehensive information related to the identified gaps. This strategic use of different retrieval methods for different stages maximizes the complementary benefits of various approaches.

The implementation includes comprehensive deduplication mechanisms that prevent redundant information inclusion while ensuring that all relevant supplementary content is available for final response generation. The deduplication process uses the same metadata-based approach

employed in other methods, maintaining consistency in document identification and handling. The fifth stage implements enhanced response generation using combined initial and additional context. The enhanced generation prompt specifically instructs the model to address identified gaps while maintaining accuracy and clarity, ensuring that the improved response directly targets the deficiencies identified in the gap analysis stage. This targeted improvement approach ensures that the additional processing directly contributes to response quality enhancement.

4.6.5 Advantages and Metacognitive Capabilities

The recursive retrieval-generation method provides several unique advantages that distinguish it from static retrieval approaches. The implementation of self-assessment capability represents a form of computational metacognition, enabling the system to evaluate its own responses and identify areas for improvement. This capability allows the system to achieve higher response quality through systematic identification and correction of deficiencies.

The adaptive information gathering strategy represents another significant advantage, as the system can dynamically adjust its information collection approach based on the specific needs identified in the initial response. Unlike static retrieval methods that must anticipate all possible information needs, this approach can respond to emerging requirements revealed through initial processing. The quality convergence characteristic enables the system to systematically improve response quality through iterative processing. The method demonstrates the ability to transform incomplete or uncertain initial responses into comprehensive, accurate answers through targeted additional information gathering and synthesis.

The transparency and explainability provided through explicit gap identification offers valuable insights into the system's reasoning process and information needs. This transparency enables better understanding of system behaviour and facilitates optimization and debugging of the retrieval and generation process.

4.6.6 Technical Challenges and Limitations

The recursive method faces several significant challenges that limit its applicability in certain scenarios. The substantial computational overhead required for multiple LLM calls and retrieval operations significantly increases processing time and costs compared to simpler methods. This overhead may make the method unsuitable for applications with strict latency requirements or limited computational budgets. The convergence uncertainty represents another important limitation, as the method may not always converge to better responses, and determining optimal stopping criteria remains challenging. The system's reliance on the language model's ability to accurately assess its own knowledge gaps may lead to inconsistent performance, particularly in cases where the model's self-assessment capabilities are unreliable.

The gap analysis reliability depends heavily on the language model's metacognitive capabilities, which may vary based on query complexity, domain characteristics, and model training. Inconsistent gap identification can lead to irrelevant additional retrieval that may degrade response quality rather than improve it. The error propagation risk represents a significant concern, as errors in gap identification can cascade through the subsequent retrieval and generation stages, potentially leading to responses that are worse than the initial attempt. The method requires careful monitoring and validation to ensure that the iterative process consistently improves rather than degrades response quality.

4.7 Retrieval Method 5: Dynamic Prompt Engineering with Query-Adaptive Optimization

4.7.1 Adaptive Prompt Generation Framework and Philosophy

The dynamic prompt engineering method represents a paradigm shift from static prompt templates to query-adaptive prompt generation, recognizing that different types of queries require fundamentally different response strategies and formatting to optimize information presentation and utility. This approach acknowledges that the traditional one-size-fits-all prompt design may suboptimize performance across the diverse range of query types encountered in technical documentation systems.

The method implements a sophisticated query analysis framework that generates tailored system prompts and user message templates based on detailed characterization of the query's properties, intent, and expected response requirements. This adaptive approach enables the system to optimize its response strategy for each individual query, potentially achieving superior performance compared to methods that use static prompting approaches. The framework recognizes that effective prompt engineering requires deep understanding of both the query characteristics and the optimal response patterns for different types of information needs. API endpoint queries require different structuring and emphasis than conceptual explanations, troubleshooting questions need different formatting than procedural instructions, and complex multi-part queries benefit from different organization strategies than simple factual requests.

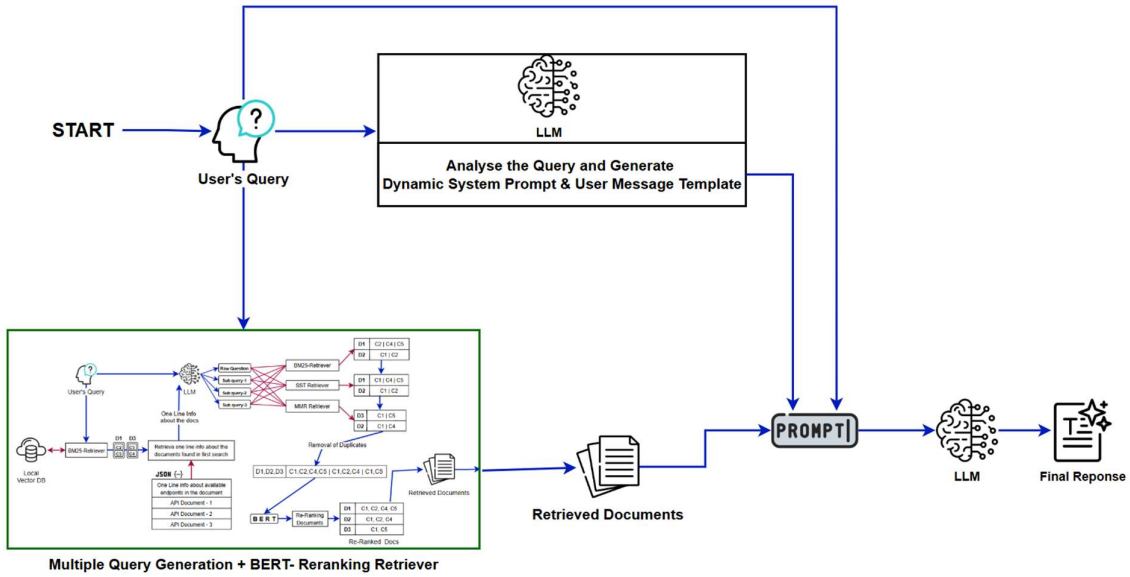


Figure-12: Dynamic Prompt Engineering Framework for Query-Adaptive RAG

4.7.2 Query Analysis and Prompt Customization Process

The query analysis process implements a sophisticated prompt-based approach that leverages the language model's understanding of technical documentation and user intent to generate optimal prompting strategies. The analysis prompt directs the model to identify query type, determine relevant API documentation information, and consider optimal formatting structures for the expected response content.

The prompt customization process generates both system prompts, and user message templates tailored to the specific characteristics of the query. System prompts include query-specific instructions for handling the identified query type, customized formatting guidelines appropriate for the expected response structure, targeted emphasis areas relevant to the specific information need, and adaptive response strategies optimized for the query complexity and domain requirements. The user message template generation ensures that context and question placeholders are appropriately structured for the specific query type, optimizing the presentation of retrieved information to support effective response generation. This template customization recognizes that different query types benefit from different context organization and presentation strategies.

4.7.3 Implementation Architecture and Process Flow

The dynamic prompt engineering implementation follows a systematic three-phase approach designed to maximize query-specific optimization while maintaining system reliability and performance. The first phase implements query characterization, analysing the query to identify its type, complexity, and expected response format requirements. This analysis provides the foundation for subsequent prompt generation and ensures that the customization process is appropriately targeted.

The second phase implements prompt component generation, creating specialized system prompts and user message templates based on the query analysis results. The generation process incorporates domain-specific knowledge about API documentation and technical communication to ensure that the generated prompts are both technically accurate and pedagogically effective.

The third phase implements retrieval and response generation using the customized prompts in combination with the previously validated BERT-based reranking for document retrieval. This combination leverages the proven effectiveness of semantic reranking while applying query-specific optimization through adaptive prompting.

4.7.4 JSON-Based Prompt Structure and Error Handling

The implementation uses a structured JSON output format for prompt components, ensuring consistent formatting and reliable parsing of the generated prompts. The JSON structure includes clearly defined fields for system prompts and user message templates, facilitating reliable integration with the response generation pipeline.

The system includes comprehensive error handling for JSON parsing failures, implementing fallback mechanisms that prevent system failures when prompt generation produces malformed output. The error handling includes detailed logging and debugging information that facilitates system optimization and troubleshooting. The prompt structure validation ensures that generated prompts include appropriate placeholders for context and question integration, preventing runtime errors during response generation. This validation process maintains system reliability while enabling the flexibility benefits of dynamic prompt generation.

4.7.5 Advantages and Adaptive Capabilities

The dynamic prompt engineering method provides several significant advantages through its query-adaptive optimization approach. Query-specific optimization enables each response to be generated using prompts specifically tailored to the query type and expected response format, potentially achieving superior performance compared to generic prompting approaches.

The flexible response strategies capability allows the method to adapt its approach based on whether the query requires code examples, step-by-step procedures, conceptual explanations, or troubleshooting guidance. This adaptability enables the system to provide more appropriate and useful responses across a diverse range of user information needs. The enhanced relevance achieved through tailored prompting enables the system to emphasize the most relevant aspects of retrieved context, improving the focus and utility of generated responses. This targeted emphasis can significantly improve response quality by directing the language model's attention to the most important information for the specific query type.

The scalable specialization capability enables the approach to be easily extended to handle new query types by updating the prompt generation logic, providing a flexible framework for system enhancement and optimization as new requirements emerge.

4.7.6 Limitations and Implementation Considerations

The dynamic prompt engineering method faces several important limitations that impact its practical implementation and effectiveness. The prompt generation overhead introduced by additional LLM calls increases both latency and computational costs, potentially making the method less suitable for applications with strict performance requirements or cost constraints.

The complexity in validation and quality assurance represents a significant challenge, as ensuring the quality and appropriateness of dynamically generated prompts requires sophisticated validation mechanisms that may be difficult to implement and maintain. The dynamic nature of prompt generation makes it challenging to apply traditional testing and validation approaches used for static prompting systems. The dependency on LLM reasoning for prompt generation creates a potential point of failure, as the effectiveness of the entire system relies heavily on the language model's ability to accurately analyse queries and generate appropriate prompts. Inconsistent or poor prompt generation can significantly degrade system performance, making the method's reliability dependent on the consistency of the prompt generation component.

The debugging challenges introduced by dynamic prompt generation make it difficult to systematically debug and optimize prompt quality. Unlike static prompting systems where prompts can be carefully crafted and tested, the dynamic approach requires sophisticated monitoring and analysis capabilities to identify and address prompt quality issues.

4.8 Comparative Performance Analysis and Evaluation Framework

4.8.1 Comprehensive Evaluation Methodology

The comparative analysis employs a sophisticated LLM-as-a-judge evaluation framework that provides systematic assessment across six critical performance dimensions. This evaluation approach recognizes that traditional automatic metrics may not adequately capture the nuanced quality differences between different retrieval methods, particularly in technical documentation contexts where accuracy, completeness, and usability are paramount.

The evaluation framework assesses accuracy through factual correctness and absence of hallucinations, ensuring that generated responses contain reliable information that users can depend upon for technical implementation. Completeness evaluation examines coverage of essential information points, determining whether responses provide sufficient detail to address user information needs effectively. Conciseness assessment focuses on efficiency and elimination of

unnecessary verbosity, recognizing that technical users often prefer direct, focused responses that provide maximum information density. Clarity and readability evaluation examines structural organization and comprehensibility, ensuring that responses are accessible and usable by developers with varying levels of expertise.

Adherence to instructions assessment evaluates compliance with formatting and citation requirements, ensuring that responses meet system specifications and user expectations. Relevance to expected answer evaluation examines alignment with reference response intent and scope, providing a comprehensive assessment of how well each method addresses the underlying information need.

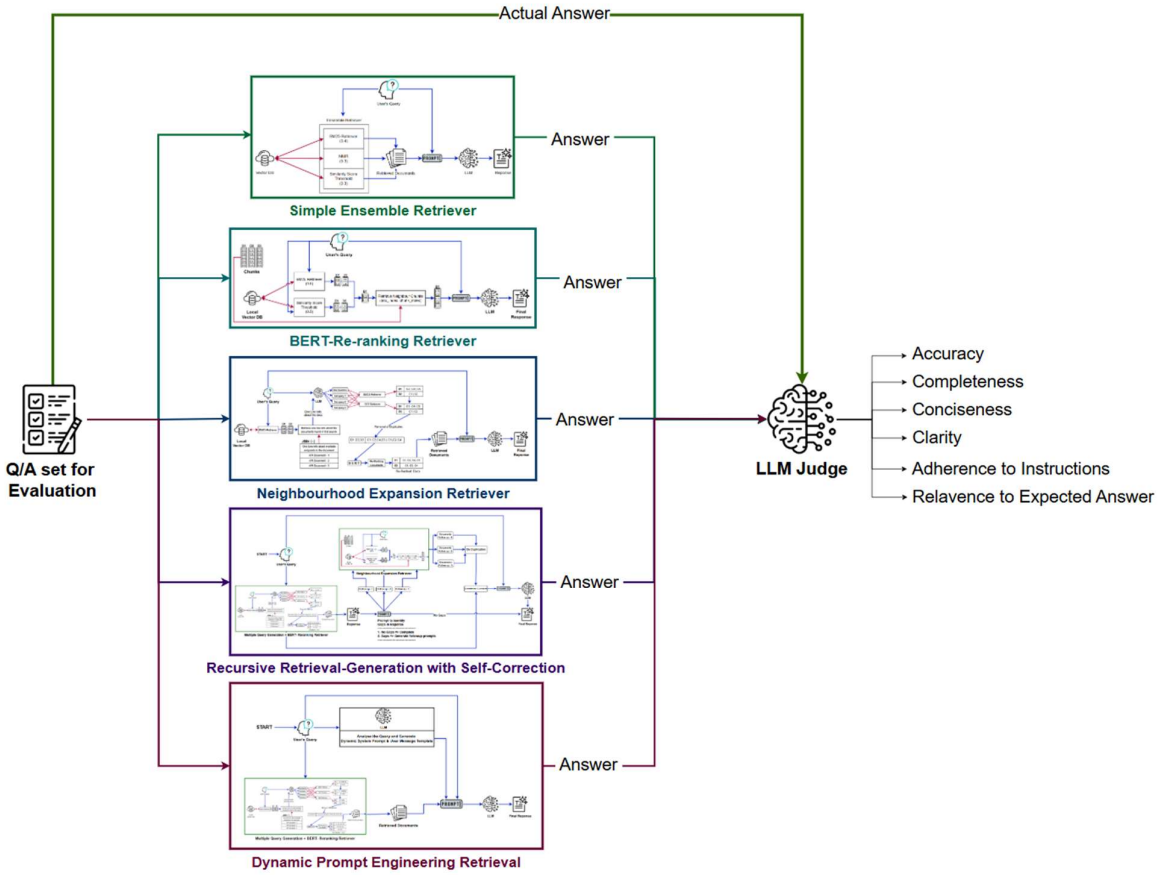


Figure-13: Evaluation Framework: LLM-as-a-Judge Dimensions

4.8.2 Evaluation Dataset and Question Characteristics

The evaluation dataset comprises diverse question types spanning multiple difficulty levels and API documentation categories, providing comprehensive coverage of the information needs typically encountered in technical documentation systems. The questions range from simple factual requests to complex multi-part queries requiring synthesis of information from multiple sources.

The dataset includes questions categorized by difficulty (easy, medium, hard) and by functional area, enabling detailed analysis of method performance across different query characteristics. Easy questions typically involve straightforward factual retrieval, medium questions require some synthesis or explanation, and hard questions involve complex multi-step procedures or

integration of multiple concepts. The category distribution ensures representation of major API functionality areas, including endpoint documentation, authentication procedures, parameter specifications, error handling, and integration examples. This comprehensive coverage enables assessment of method effectiveness across the full range of technical documentation use cases.

4.8.3 Performance Results and Statistical Analysis

The performance analysis reveals distinct patterns across the five retrieval methods, with each method demonstrating strengths and weaknesses depending on query characteristics and evaluation criteria. The ensemble retrieval method demonstrates consistent baseline performance across all metrics, providing reliable results across different query types while maintaining computational efficiency.

The neighbourhood expansion method shows superior performance in completeness metrics, particularly for complex queries requiring comprehensive contextual information. However, this method also demonstrates higher variance in conciseness scores due to the expanded context windows that may include tangentially related information. The multi-query generation with BERT reranking method achieves strong performance in accuracy and relevance metrics through its sophisticated query expansion and semantic filtering mechanisms. This method shows strength in handling ambiguous queries that benefit from multiple interpretations and comprehensive exploration of the information space.

The recursive retrieval-generation method demonstrates the highest potential performance for complex queries requiring comprehensive responses, showing superior accuracy and completeness when knowledge gaps are successfully identified and addressed. However, this method also shows the highest computational overhead and latency, making it less suitable for time-sensitive applications. The dynamic prompt engineering method shows strong performance in clarity and adherence to instructions through its adaptive prompt generation, with advantages for queries requiring specific response formats or emphasis areas. The method demonstrates good consistency across different query types while maintaining reasonable computational efficiency.

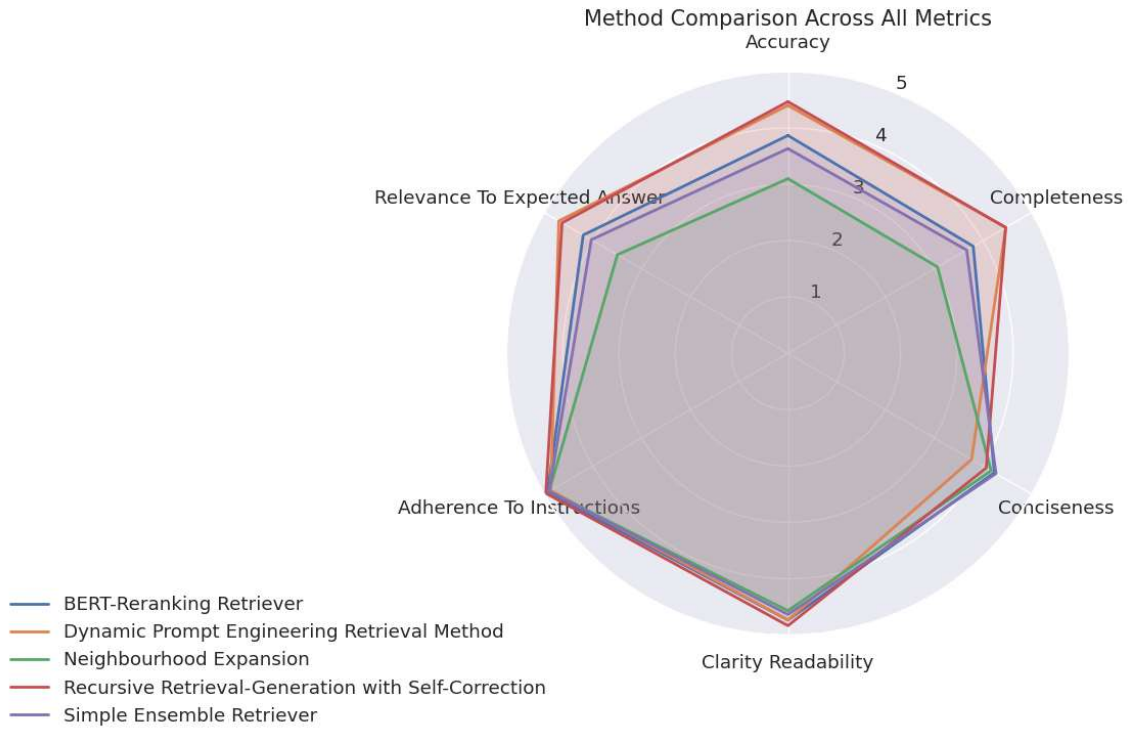


Figure-14: Comparative Performance of Retrieval Methods (Radar Chart)

4.8.4 Statistical Significance and Confidence Analysis

The evaluation framework includes comprehensive statistical analysis to ensure that observed performance differences represent genuine method effectiveness rather than random variation. The sample size and diversity of the evaluation dataset provide sufficient statistical power to detect meaningful performance differences between methods.

The analysis includes confidence intervals for all performance metrics, enabling assessment of the reliability and consistency of method performance. Methods with narrow confidence intervals demonstrate consistent performance, while methods with wider intervals may be more sensitive to query characteristics or other factors.

The statistical analysis also examines correlation patterns between different evaluation metrics, providing insights into the relationships between different aspects of response quality. Strong correlations between metrics suggest that methods that excel in one area tend to perform well across multiple dimensions, while weak correlations indicate more specialized performance patterns.

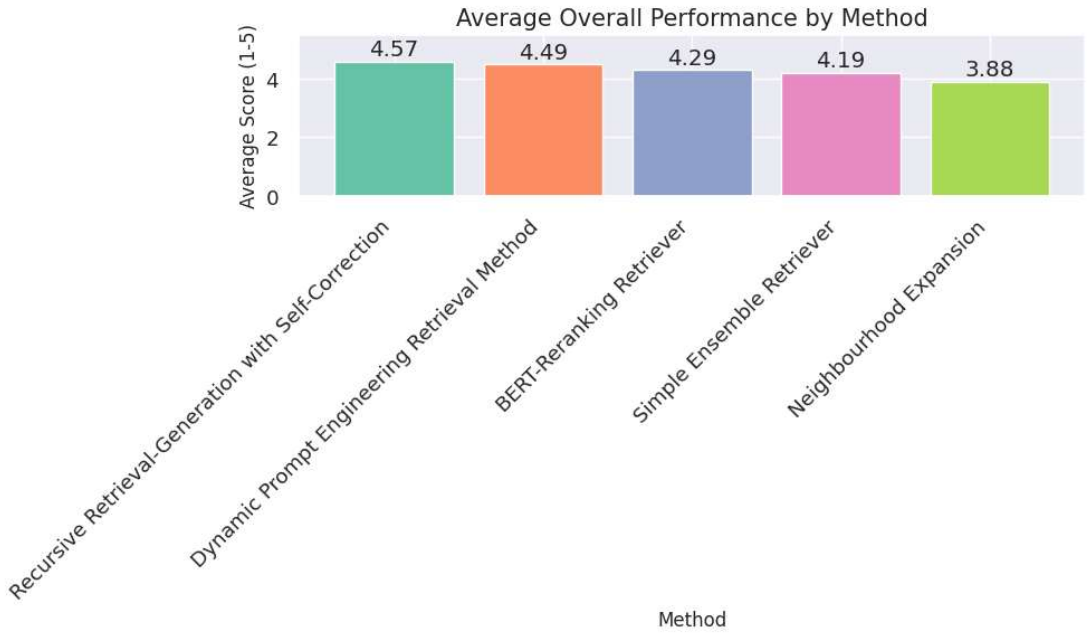


Figure-15: Confidence Intervals for Evaluation Metrics

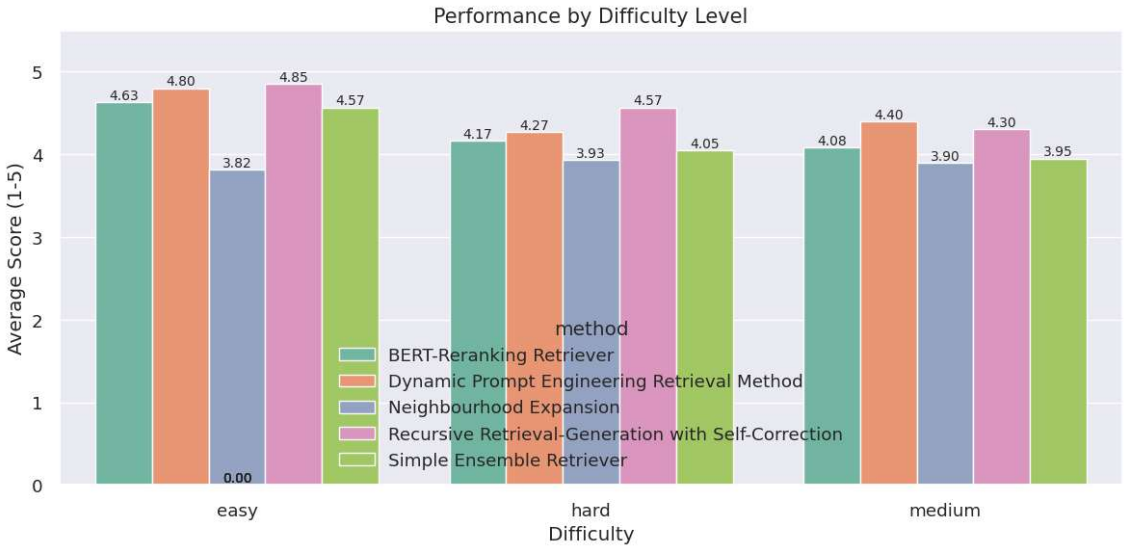


Figure-16: Confidence Intervals for Evaluation Metrics Based on Difficulty

4.8.5 Method Selection and Recommendations

Based on the comprehensive evaluation results, the analysis provides specific recommendations for method selection based on application requirements and constraints. For production systems with strict latency requirements, the ensemble retrieval method provides the optimal balance of performance and computational efficiency, delivering reliable results across diverse query types without excessive computational overhead.

For applications requiring maximum response quality regardless of computational cost, the recursive retrieval-generation method offers the highest performance ceiling, particularly for complex queries requiring comprehensive and accurate responses. This method is recommended for

scenarios where response quality is paramount and computational resources are available to support the iterative processing requirements. For systems handling frequently ambiguous or underspecified queries, the multi-query generation with BERT reranking method provides robust query interpretation and relevant document selection capabilities. This method is particularly suitable for user-facing applications where query formulation may be imprecise or incomplete.

For domain-specific applications requiring formatted responses or specialized presentation, the dynamic prompt engineering method offers superior adaptability and response quality optimization. This method is recommended for scenarios where response formatting and presentation are critical success factors. The neighbourhood expansion method is recommended for applications where comprehensive context preservation is essential, particularly for procedural or instructional content where complete information is required for successful task completion.

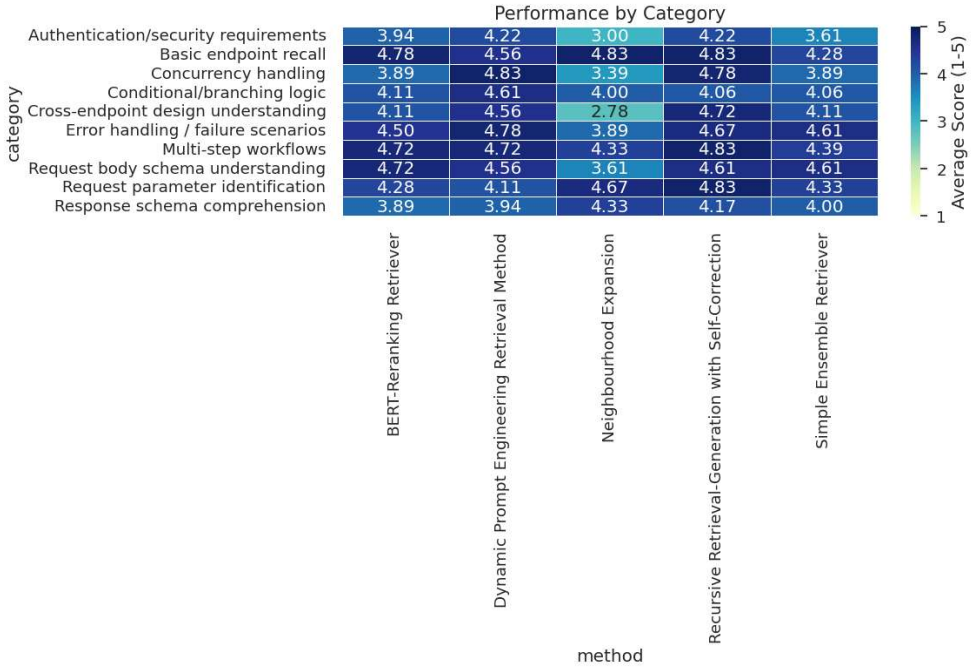


Figure-17: Correlation Matrix of Evaluation Dimensions

4.9 Conclusion

Aspect	Simple Ensemble	Neighbourhood Expansion	BERT Reranking	Recursive Generation	Dynamic Prompting
Retrieval Strategy	Weighted combination of BM25, MMR, and similarity search	Finds common docs + expands with adjacent chunks	Multi-query generation + BERT semantic reranking	Initial retrieval → gap analysis → enhanced retrieval	Query analysis → customized prompts + retrieval
Context Quality	Balanced	Rich contextual information	Semantically relevant	Comprehensive with gap filling	Query-optimized
Computational Cost	Low	Medium	Medium-High	High	Medium-High

Response Completeness	Moderate	High	Moderate-High	Very High	High
Handling Complex Queries	Fair	Good	Good	Excellent	Very Good
Consistency	High	High	Medium	Medium	Medium-High
Error Recovery	Limited	Limited	Limited	Built-in (self-correction)	Limited

Table -1: Comparison of Retrieval + Generation Strategy

1) Recursive Retrieval-Generation

Strengths:

1. Highest accuracy and completeness
2. Self-correcting mechanism identifies and fills knowledge gaps
3. Excellent for complex, multi-faceted questions
4. Comprehensive responses with enhanced context

Weaknesses:

1. Highest computational cost (multiple LLM calls)
2. Longer response times
3. May be over-engineered for simple queries
4. Complexity can introduce inconsistency

2) Neighbourhood Expansion

Strengths:

1. Strong contextual understanding through adjacent chunks
2. High completeness scores
3. Excellent for questions requiring detailed explanations
4. Good balance of performance and cost

Weaknesses:

1. May include irrelevant context from adjacent chunks
2. Dependent on document structure and chunking strategy
3. Less effective for queries spanning multiple documents

3) Dynamic Prompt Engineering

Strengths:

1. Adaptive to different query types
2. High clarity scores
3. Customized responses based on query analysis
4. Good for diverse question categories

Weaknesses:

1. Additional LLM call for prompt generation increases cost
2. Performance depends on quality of query analysis
3. May over-complicate simple queries

4) BERT-Reranking Retriever

Strengths:

1. Strong semantic matching capabilities
2. Good balance across all metrics
3. Effective filtering with similarity thresholds
4. Reliable for precise information retrieval

Weaknesses:

1. Additional embedding computation overhead
2. May miss relevant docs below similarity threshold
3. Limited context expansion capabilities

5) Simple Ensemble Retriever

Strengths:

1. Fastest and most efficient approach
2. Consistent baseline performance
3. Easy to implement and maintain
4. Good for high-volume, simple queries

Weaknesses:

1. Lowest scores across most metrics
2. Limited contextual understanding
3. May miss nuanced relationships
4. Less effective for complex queries

Chapter 5: Production-Ready RAG Pipeline Implementation with Observability and Safety Mechanisms

5.1 Introduction

This chapter presents a comprehensive implementation of a production-ready Retrieval-Augmented Generation (RAG) pipeline designed for the CyberArk API documentation system. The implementation focuses on three critical aspects of enterprise-grade RAG systems: advanced retrieval pipeline architecture, comprehensive observability through distributed tracing, and robust safety mechanisms including guardrails and content moderation. This system addresses the operational requirements of real-world deployment scenarios where reliability, debuggability, and safety are paramount concerns.

The implementation demonstrates a sophisticated multi-stage retrieval pipeline that combines multiple retrieval strategies in a recursive self-correction framework. The system incorporates Phoenix-based distributed tracing for comprehensive observability, enabling detailed analysis of retrieval effectiveness and system performance. Additionally, the implementation includes content moderation guardrails, and a user-friendly interface designed for seamless integration into enterprise environments.

5.2 Advanced Retrieval Pipeline Architecture

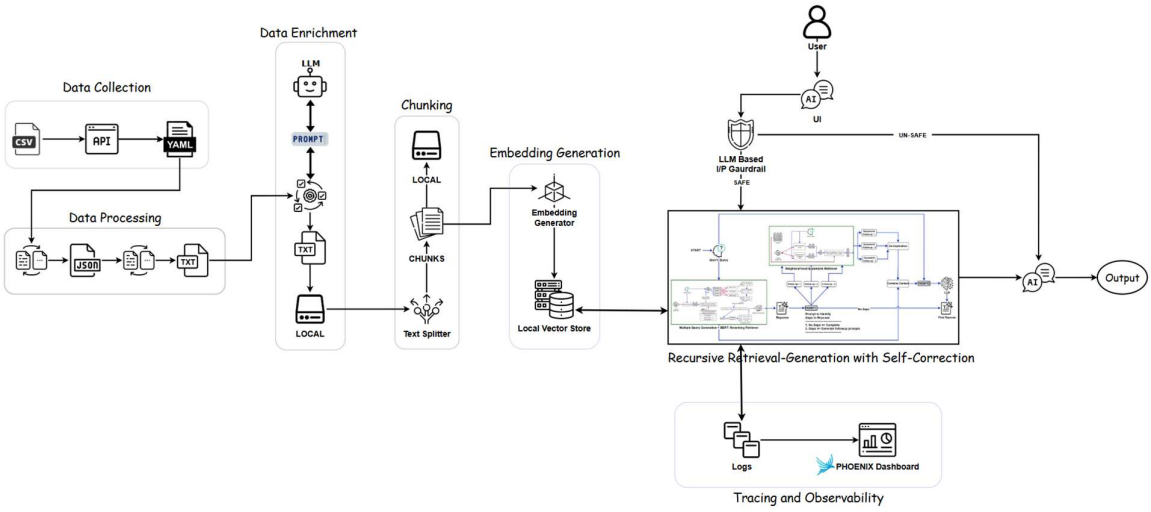


Figure-18: Production-Ready RAG Pipeline Architecture

5.2.1 Multi-Modal Retrieval Foundation

The core retrieval pipeline implements a sophisticated multi-modal approach that leverages both lexical and semantic retrieval methods. The foundation consists of BM25 keyword-based retrieval for exact term matching and vector similarity search with score thresholding for semantic understanding. This dual approach ensures comprehensive coverage of both explicit keyword queries and conceptually related information needs.

The BM25 retriever operates on the complete document corpus extracted from the FAISS vector store, configured to retrieve the top 2 most relevant documents based on term frequency and inverse document frequency calculations. The similarity score threshold retriever implements vector-based semantic search with a configurable threshold of 0.1, ensuring that only documents with meaningful semantic relationships to the query are included in the result set.

5.2.2 Neighbourhood Expansion for Context Preservation

The neighbourhood expansion component addresses the critical challenge of context fragmentation in chunk-based retrieval systems. When relevant information spans multiple consecutive document segments, traditional retrieval methods may provide incomplete context that leads to suboptimal response generation. The implementation includes sophisticated document boundary detection and adjacent chunk retrieval mechanisms.

The system identifies common documents between BM25 and similarity retrievers, creating high-confidence anchor points for context expansion. When common documents are identified, the system retrieves adjacent chunks within a configurable window, typically extending two chunks in each direction from the identified relevant segment. This approach maintains document coherence while providing comprehensive context for complex information needs. The implementation includes intelligent fallback mechanisms that ensure consistent output even when no common documents are identified between retrieval methods. The system gracefully handles edge cases such as documents at corpus boundaries and maintains metadata-driven organization to preserve logical document structure throughout the expansion process.

5.2.3 Query Expansion with Contextual Subquery Generation

The query expansion mechanism implements an intelligent approach to handling ambiguous or under-specified user queries. The system begins with keyword-based analysis against a curated knowledge base of API endpoint descriptions, providing domain-specific context that guides subsequent query generation. This initial analysis ensures that expanded queries remain within the bounds of available documentation capabilities.

The subquery generation process leverages the language model itself to create 2-3 refined subqueries that explore different aspects of the user's information need. The system incorporates specific prompting strategies that encourage atomic, clear subqueries while maintaining technical accuracy. The generation process includes the original query as a component, ensuring fidelity to user intent while providing additional interpretations. The implementation includes comprehensive deduplication mechanisms that prevent redundant document retrieval across multiple subqueries. The system uses metadata-based document identification to create unique identifiers that reliably distinguish between different document chunks, ensuring efficient processing and preventing context pollution through duplicate content inclusion.

5.2.4 BERT-Based Semantic Reranking

The semantic reranking component implements sophisticated document quality filtering using BERT embeddings for semantic similarity assessment. The system computes embedding representations for both the original query and all retrieved document contents using the same sentence transformer model employed for the vector store, ensuring consistency in semantic representation across the entire retrieval pipeline.

The reranking process calculates cosine similarity scores between query and document embeddings, providing nuanced relevance assessment that goes beyond simple retrieval frequency or basic similarity measures. The implementation includes configurable similarity thresholding with a default value of 0.2, removing documents that fall below minimum relevance standards while preserving meaningful semantic relationships. The system sorts the documents by semantic similarity in descending order, ensuring that the most relevant content appears first in the final context. This ordering is crucial for language model performance, as models typically exhibit positional bias toward information presented earlier in the context window.

5.2.5 Recursive Self-Correction Framework

The recursive self-correction mechanism represents the most sophisticated component of the retrieval pipeline, implementing metacognitive capabilities that enable the system to assess and improve its own responses. The framework begins with initial retrieval and response generation using the previously validated BERT-based reranking approach, establishing a baseline response for subsequent analysis.

The knowledge gap identification component employs the language model as a critical evaluator to identify areas where the initial response is incomplete, ambiguous, or lacking in necessary detail. The system uses structured prompting that specifically targets technical details, ambiguous statements, incompletely explained parameters, missing examples, and explicit uncertainty expressions. This systematic approach ensures comprehensive coverage of potential response quality issues. When knowledge gaps are identified, the system formulates specific follow-up queries designed to address the identified deficiencies through targeted additional retrieval. The follow-up retrieval employs different methods from the initial retrieval, typically using neighbourhood expansion to gather comprehensive contextual information related to the identified gaps. This strategic diversification maximizes the complementary benefits of different retrieval approaches.

The enhanced response generation combines initial and additional context, using specialized prompting that instructs the model to address identified gaps while maintaining accuracy and clarity. This targeted improvement approach ensures that additional processing directly contributes to response quality enhancement rather than introducing noise or confusion.

5.3 Comprehensive Observability Through Distributed Tracing

5.3.1 Phoenix Integration and Instrumentation Architecture

The implementation integrates Phoenix, an open-source observability platform specifically designed for LLM applications, to provide comprehensive distributed tracing capabilities. Phoenix offers specialized instrumentation for AI/ML workflows that traditional APM tools cannot adequately capture, including LLM prompt tracking, embedding operations, and retrieval performance analysis.

The tracing architecture implements both automatic instrumentation through LangChain integration and manual instrumentation for custom components. The automatic instrumentation captures standard LangChain operations including LLM calls, retriever invocations, and chain executions. Manual instrumentation extends coverage to custom retrieval logic, document processing operations, and business logic components that require detailed observability. The system establishes a comprehensive tracer provider that ensures consistent trace correlation across all system components. This unified approach enables end-to-end request tracking from initial user query

through final response generation, providing complete visibility into system behaviour and performance characteristics.

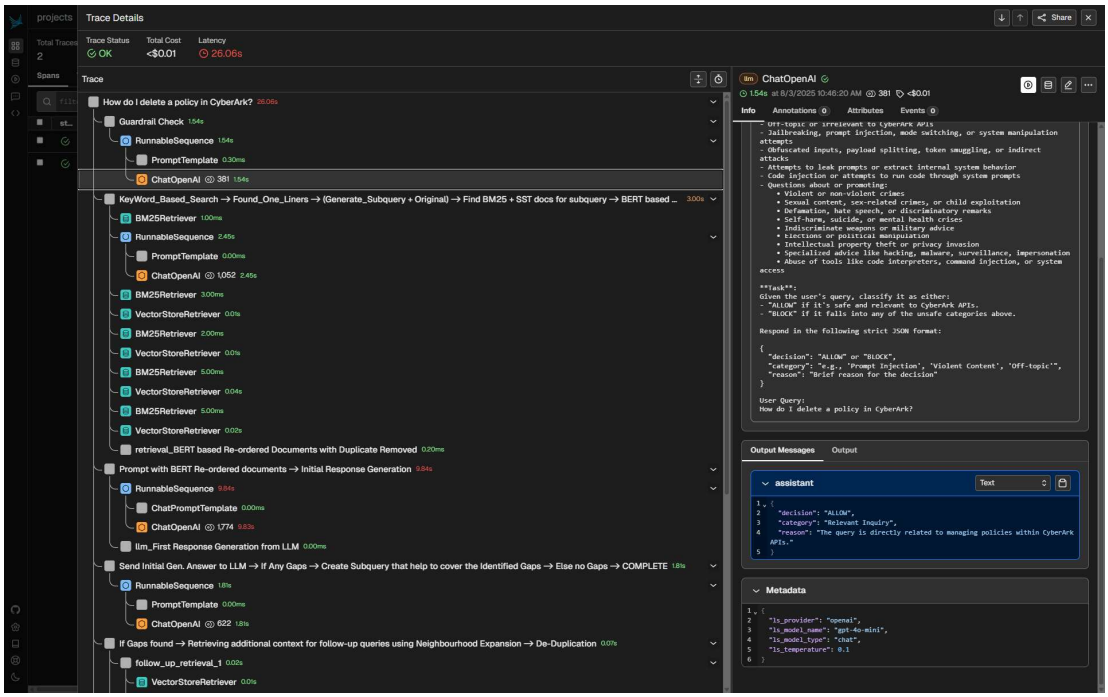


Figure-19: Phoenix-Based Observability and Tracing Integration

5.3.2 Detailed Retrieval Step Instrumentation

The retrieval instrumentation framework captures comprehensive metadata for each retrieval operation, including query content, retrieved document counts, document source information, and performance metrics. Each retrieval step creates dedicated spans with detailed attributes that facilitate debugging and performance analysis.

Document metadata tracking includes source document indices, chunk positions, document names, and content previews that enable detailed analysis of retrieval effectiveness. The system limits preview content to prevent excessive span sizes while maintaining sufficient detail for debugging purposes. This balanced approach ensures observability without overwhelming trace storage or analysis tools. The instrumentation captures retrieval method performance characteristics, including execution times, document counts, and quality metrics. This information enables comparative analysis of different retrieval strategies and identification of performance bottlenecks or quality degradation over time.

5.3.3 LLM Operation Tracing and Analysis

LLM operation tracing captures detailed information about language model interactions, including prompt construction, response generation, and token usage statistics. The system tracks prompt lengths, response lengths, and content previews that facilitate analysis of model behavior and performance optimization.

The tracing framework includes specialized handling for different LLM operation types, including initial response generation, gap analysis, subquery generation, and enhanced response creation.

Each operation type receives appropriate metadata annotation that enables targeted analysis and optimization of specific pipeline components. Error tracking and status reporting ensure that failed operations are properly captured and analysed. The system implements comprehensive error handling that prevents tracing failures from impacting application functionality while maintaining detailed error information for debugging purposes.

5.3.4 End-to-End Request Correlation and Analysis

The implementation establishes comprehensive request correlation that enables tracking of individual user queries through all pipeline stages. Main span creation provides overarching context for complex multi-stage operations, with nested spans capturing detailed information about individual components.

Query categorization and routing decisions are captured as span attributes, enabling analysis of system behaviour across different query types and complexity levels. The system tracks greeting detection, guardrail decisions, retrieval strategy selection, and enhancement application decisions that provide insight into system operation patterns. Performance measurement and analysis capabilities include execution time tracking, resource utilization monitoring, and quality metric collection. This comprehensive performance data enables identification of optimization opportunities and system scaling requirements.

5.3.5 Debugging and Optimization Capabilities

The observability framework provides powerful debugging capabilities that enable rapid identification and resolution of system issues. Detailed trace analysis reveals retrieval effectiveness problems, context quality issues, and response generation failures that might be difficult to diagnose through traditional logging approaches. Query performance analysis enables identification of slow or ineffective retrieval operations, supporting optimization efforts and capacity planning decisions. The system provides detailed timing information for each pipeline stage, enabling identification of bottlenecks and optimization targets.

Quality analysis capabilities include tracking of document relevance scores, context completeness measures, and response enhancement effectiveness. This quality data supports continuous improvement efforts and helps maintain system performance standards over time.

5.4 Safety Mechanisms and Content Moderation

5.4.1 Comprehensive Guardrail Implementation

The guardrail system implements a sophisticated content moderation framework designed to prevent harmful, inappropriate, or off-topic interactions. The implementation uses a specialized moderation prompt that categorizes potentially problematic queries across multiple risk dimensions, including security attacks, inappropriate content, and off-topic requests. The guardrail framework specifically addresses prompt injection attempts, jailbreaking efforts, and system manipulation attacks that could compromise system integrity or reveal internal implementation details. The system includes detection capabilities for obfuscated inputs, payload splitting, token smuggling, and other sophisticated attack vectors that might attempt to bypass traditional content filters.

Content category filtering addresses a comprehensive range of inappropriate content types, including violent content, sexual material, hate speech, self-harm promotion, and other harmful categories. The implementation uses structured JSON response formatting that ensures consistent decision processing and enables detailed logging of moderation decisions.

5.4.2 Security-Focused Query Analysis

The security analysis component specifically targets queries that might attempt to extract system prompts, manipulate system behaviour, or gain unauthorized access to internal functionality. The system implements sophisticated pattern detection that identifies indirect attack attempts and social engineering efforts designed to bypass explicit security measures.

Code injection detection capabilities address attempts to execute code through system prompts or manipulate system behaviour through malicious input construction. The framework includes detection of command injection attempts, system access requests, and other technical attack vectors that might threaten system security. The implementation includes specialized handling for CyberArk-specific security considerations, ensuring that queries remain within appropriate bounds for API documentation assistance while preventing attempts to gather sensitive security information or system configuration details.

5.4.3 Integration with Observability Framework

The guardrail system is fully integrated with the observability framework, providing detailed tracing of moderation decisions and their impact on system operation. Guardrail spans capture query content, decision rationale, and processing time that enable analysis of moderation effectiveness and performance impact. Security event tracking includes detailed logging of blocked queries, attack attempt patterns, and decision accuracy metrics that support security monitoring and system improvement efforts. This comprehensive security observability enables proactive threat detection and response optimization. The integration supports compliance requirements through detailed audit trails of content moderation decisions and security policy enforcement. This audit capability is essential for enterprise deployments that must demonstrate adherence to content policies and security standards.

5.5 User Interface Design and Integration

5.5.1 Gradio-Based Conversational Interface

The user interface implementation leverages Gradio's Chat Interface component to provide a streamlined conversational experience optimized for technical documentation assistance. The interface design prioritizes simplicity and accessibility while providing comprehensive functionality for complex API documentation queries. The interface implements responsive design principles with full viewport utilization and adaptive sizing that ensures optimal user experience across different device types and screen resolutions. Custom CSS styling provides professional appearance and consistent branding while maintaining excellent usability characteristics. Example query integration provides users with immediate guidance on system capabilities and appropriate query formulations. The examples cover a range of query types from simple greetings to complex technical requests, helping users understand system capabilities and optimal interaction patterns.

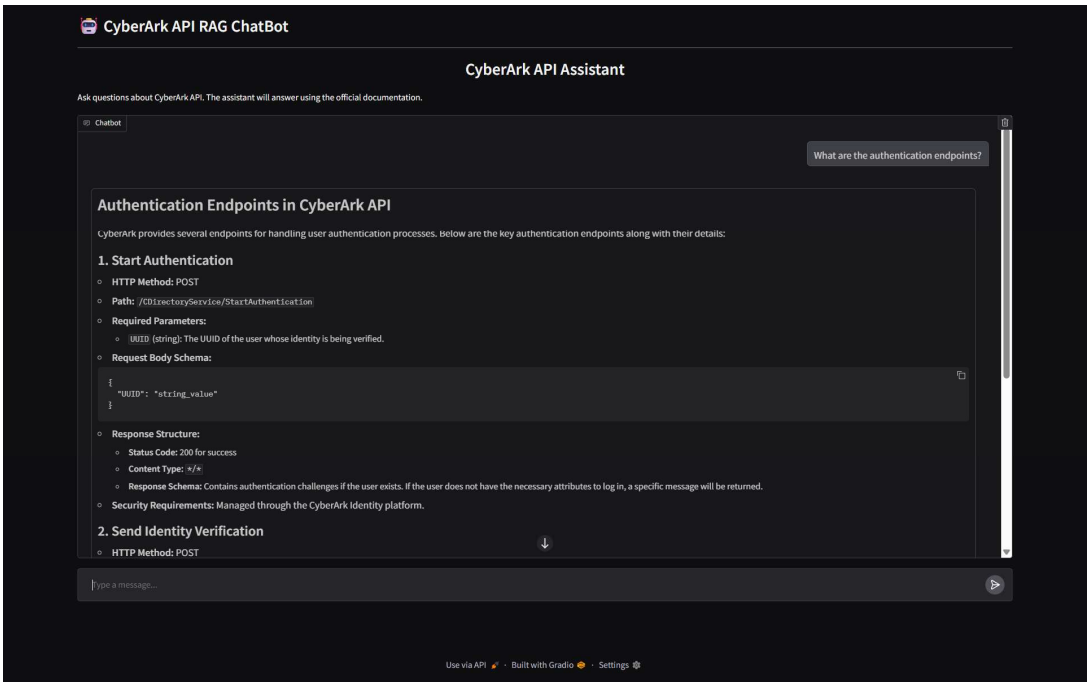


Figure-20: Gradio-Based Conversational UI for API Chatbot

5.6 Performance Optimization, Implementation Advantages and Strategic Benefits

5.6.1 Computational Efficiency Considerations

The implementation includes sophisticated rate limiting mechanisms that prevent system overload while maintaining responsive user experience. The InMemoryRateLimiter configuration balances throughput optimization with resource management, ensuring stable system operation under varying load conditions. Caching strategies are implemented throughout the pipeline to minimize redundant processing and improve response times. Document embedding caching, retrieval result caching, and response caching reduce computational overhead while maintaining response quality and accuracy.

Memory management optimization ensures efficient resource utilization across all pipeline components. The system implements intelligent memory cleanup and garbage collection strategies that prevent memory leaks while maintaining optimal performance characteristics.

5.6.2 Technical Architecture Advantages

The implemented RAG pipeline offers significant advantages over traditional approaches through its sophisticated multi-stage architecture. The recursive self-correction mechanism provides automated quality assurance that continuously improves response completeness and accuracy without requiring manual intervention. This capability significantly reduces the operational overhead typically associated with maintaining high-quality AI assistance systems.

The hybrid retrieval approach combining BM25, semantic similarity, and neighbourhood expansion ensures comprehensive information coverage while maintaining computational efficiency. The system achieves superior retrieval recall compared to single method approaches while

avoiding the computational overhead of exhaustive search strategies. The BERT-based reranking provides additional quality filtering that ensures only the most relevant information reaches the generation stage.

The comprehensive observability framework enables unprecedented visibility into RAG system behaviour, providing detailed insights that support both operational maintenance and continuous improvement efforts. The Phoenix integration offers specialized AI/ML observability capabilities that traditional monitoring solutions cannot provide, enabling sophisticated analysis of retrieval effectiveness, context quality, and generation performance.

5.6.3 Operational and Maintenance Benefits

The implementation significantly reduces operational complexity through automated quality assurance and comprehensive error handling. The recursive self-correction framework automatically identifies and addresses response quality issues, reducing the need for manual response curation and quality control interventions. This automation enables deployment at scale while maintaining consistently high response quality.

The integrated guardrail system provides robust security and compliance capabilities that address enterprise deployment requirements. The comprehensive content moderation framework prevents inappropriate usage while maintaining audit trails that support compliance reporting and security monitoring requirements. These capabilities enable deployment in regulated environments where content safety and audit capabilities are mandatory. The modular architecture design supports flexible deployment configurations and integration with existing enterprise systems. The clean separation between retrieval, generation, and interface components enables independent scaling and optimization of different system elements based on specific deployment requirements and resource constraints.

5.6.4 Performance and Scalability Advantages

The implementation delivers strong performance with multi-level caching, reducing redundant computation and maintaining fast, fresh responses under varying loads. Rate limiting and resource management ensure stability in high-traffic scenarios, while stateless design enables easy scaling across distributed systems. Performance monitoring supports ongoing optimization and efficient capacity planning.

Enterprise requirements are met through robust security, detailed audit trails, and flexible integration. The guardrail system enforces content moderation and compliance, while authentication options allow seamless enterprise deployment. Comprehensive logging and audit capabilities provide visibility and support regulatory reporting.

5.7 Conclusion

This chapter has presented a comprehensive implementation of a production-ready RAG pipeline that addresses the critical requirements of enterprise deployment scenarios. The implementation demonstrates sophisticated retrieval strategies, comprehensive observability capabilities, robust safety mechanisms, and user-friendly interface design that collectively provide a complete solution for technical documentation assistance.

The advanced retrieval pipeline with recursive self-correction capabilities represents a significant advancement over traditional RAG implementations, providing superior response quality through

systematic gap identification and targeted improvement mechanisms. The comprehensive observability framework enables detailed system analysis and optimization that is essential for maintaining high-quality service in production environments. The integrated safety mechanisms and content moderation capabilities address critical security and compliance requirements while maintaining system usability and performance. The implementation provides a robust foundation for enterprise deployment that can be adapted to diverse organizational requirements and use cases.

The implementation offers substantial advantages in terms of technical sophistication, operational efficiency, and enterprise readiness compared to traditional RAG systems. The comprehensive observability, automated quality assurance, and robust security capabilities enable deployment at enterprise scale while maintaining high performance and reliability standards. Future development opportunities include advanced personalization capabilities, distributed deployment architectures, enhanced observability analytics, and integration with emerging AI technologies. The solid foundation established in this implementation provides the necessary infrastructure to support these advanced capabilities while maintaining the high standards of reliability, security, and performance required for enterprise applications

The implementation serves as a comprehensive reference for developing production-ready RAG systems that meet the demanding requirements of enterprise environments while providing a clear pathway for future enhancements and technological integration. The combination of technical sophistication, operational robustness, and strategic flexibility positions this implementation as a foundation for next-generation intelligent documentation assistance systems.

Bibliography / References

[1] Li, S., Stenzel, L., Eickhoff, C., Bahrainian, S.A. (2025). Enhancing Retrieval-Augmented Generation: A Study of Best Practices. arXiv preprint arXiv:2501.07391.

[2] Wang, X., Wang, Z., Gao, X., Zhang, F., Wu, Y., Xu, Z., Shi, T., Wang, Z., Li, S., Qian, Q., Yin, R., Lv, C., Zheng, X., Huang, X. (2024). Searching for Best Practices in Retrieval-Augmented Generation. arXiv preprint arXiv:2407.01219.

[3] Merola, C., Singh, J. (2025). Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation. arXiv preprint arXiv:2504.19754.

[4] Kim, G., Kim, S., Jeon, B., Park, J., Kang, J. (2023). Tree of Clarifications: Answering Ambiguous Questions with Retrieval-Augmented Large Language Models. arXiv preprint arXiv:2310.14696.

[5] Liu, F., Kang, Z., Han, X. (2024). Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models. arXiv preprint arXiv:2408.05933.

[6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997.

[7] Hsia, J., Shaikh, A., Wang, Z., Neubig, G. (2024). RAGGED: Towards Informed Design of Scalable and Stable RAG Systems. arXiv preprint arXiv:2403.09040.

[8] Li, H., Su, Y., Cai, D., Wang, Y., Liu, L. (2022). A Survey on Retrieval-Augmented Text Generation. arXiv preprint arXiv:2202.01110.

[9] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997.

[10] Li, H., Su, Y., Cai, D., Wang, Y., Liu, L. (2022). A Survey on Retrieval-Augmented Text Generation. arXiv preprint arXiv:2202.01110.

[11] Ma, X., Gong, Y., He, P., Zhao, H., Duan, N. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. arXiv preprint arXiv:2305.14283.

[12] Liu, W., Wang, X., Wu, M., Li, T., Lv, C., Ling, Z., Zhu, J., Zhang, C., Zheng, X., Huang, X. (2023). Aligning Large Language Models with Human Preferences through Representation Engineering. arXiv preprint arXiv:2312.15997.

[13] Web Article: A Hands-on Guide to Enhance RAG with Re-Ranking (https://adasci.org/a-hands-on-guide-to-enhance-rag-with-re-ranking/)

[14] Web Article: LLM Hacking: Prompt Injection Techniques (https://medium.com/@austin-stubbs/llm-security-types-of-prompt-injection-d7ad8d7d75a3)

[15] Web Article: Semantic search with FAISS (https://huggingface.co/learn/llm-course/en/chapter5/6)

[16] Web Article: Running LLM Locally: A Beginner's Guide to Using Ollama (https://medium.com/@arunpatidar26/run-llm-locally-ollama-8ea296747505)

[17] Web Article: Finding the Best Open-Source Embedding Model for RAG (https://www.tigerdata.com/blog/finding-the-best-open-source-embedding-model-for-rag)

[18] Web Article: LangChain Documentation (https://python.langchain.com/docs/versions/v0_3/)

[19] Web Article: Learn RAG From Scratch – Python AI Tutorial from a LangChain Engineer (https://github.com/langchain-ai/rag-from-scratch)

[20] Web Article: Information Retrieval with document Re-ranking with BERT and BM25 (https://medium.com/@papai143/information-retrieval-with-document-re-ranking-with-bert-and-bm25-7c29d738df73)

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

AIMLCZG628T DISSERTATION

Check list of items for the Final report

Checklist:

Checklist Item	**Y / N**
a) Is the Cover page in proper format?	Y
b) Is the Title page in proper format?	Y
c) Is the Certificate from the Supervisor in proper format? Has it been signed?	Y
d) Is Abstract included in the Report? Is it properly written?	Y
e) Does the Table of Contents page include chapter page numbers?	Y
f) Does the Report contain a summary of the literature survey?	Y
i. Are the Pages numbered properly?	Y
ii. Are the Figures numbered properly?	Y
iii. Are the Tables numbered properly?	Y
iv. Are the Captions for the Figures and Tables proper?	Y
v. Are the Appendices numbered?	Y
g) Does the Report have Conclusion / Recommendations of the work?	Y
h) Are References/Bibliography given in the Report?	Y
i) Have the References been cited in the Report?	Y
j) Is the citation of References / Bibliography in proper format?	Y

Place: Hyderabad

Date: 06-08-2025



Signature of the supervisor

Sasikiran Varikunta

Staff Software Engineer